ANSWERS, CODE AND CHARTS

Case Study Resolutions

Every question solved on the real case data: code (Python, T-SQL, Excel or Tableau steps), outputs, charts and business insights.

AnalytixLabs · Advanced Certification in Data Science & AI (IIT), June 2026 batch

Interactive version (every tab clickable):
https://claude.ai/artifact/GtUBqgmq7C6gvyYa7LjhEw

Contents

Python for Data Science

- Python Exercise 1: Basic Python Programming (20 questions)
- Python Exercise 2: Data Importing, Manipulation, Analysis and Visualisation (90 questions)
- Python Case Study 1: Customer Analysis for Retail
- Python Case Study 2: Exploratory Analysis of Credit Card Data
- Python Case Study 3: EDA and Hypothesis Testing on Insurance Claims
- Python Case Study 4: Visualisations on Pharma Sales Data
- Basic Statistics Case Study: Hypothesis Testing (5 business problems)
- Python End-to-End Case Study: Marketing Analytics for an E-commerce Marketplace

Excel (Data Visualization & Analytics)

- Excel Case Study: Sportsmen Roster (Data Cleaning, Analysis, Report)
- Excel Case Study: Consumer Complaint Analysis

SQL (Data Visualization & Analytics)

- SQL Case Study (Basic): Retail Data Analysis
- SQL Case Study (Advanced): Mobile Manufacturer Data Analysis

Tableau

- Tableau Case Study: Laptop Sales Strategy (London retail chain)
- Tableau Case Study: Sports Equipment Retailer

Python Exercise 1: Basic Python Programming (20 questions)

Twenty short programs covering operators, input/output, loops, data types, list comprehensions, lambda/filter and user-defined functions. Where a question asks for console input, the solution uses a fixed sample value so the output is shown; replace it with input() in Jupyter.

Data

No data files.

Tool and module

Python (core)
Python for Data Science

1 Output of 5 + 4 * 9 % (3 + 1) / 6 - 1?

PYTHON

```python
print(5 + 4 * 9 % (3 + 1) / 6 - 1)
# 4*9=36 -> 36 % 4 = 0 -> 0/6 = 0.0 -> 5 + 0.0 - 1 = 4.0
```

RESULT ON THE REAL CASE DATA

```
4.0
```

2 Check if a number is odd or even (number taken as input).

PYTHON

```python
n = 7   # n = int(input('Enter a number: '))
print(n, 'is', 'Even' if n % 2 == 0 else 'Odd')
```

RESULT ON THE REAL CASE DATA

```
7 is Odd
```

3 Print the multiplication table of a number.

PYTHON

```python
n = 7   # n = int(input())
for i in range(1, 11):
```

```
    print(f'{n} x {i:2d} = {n*i}')
```

RESULT ON THE REAL CASE DATA

```
7 x  1 = 7
7 x  2 = 14
7 x  3 = 21
7 x  4 = 28
7 x  5 = 35
7 x  6 = 42
7 x  7 = 49
7 x  8 = 56
7 x  9 = 63
7 x 10 = 70
```

4 Numbers between 2000 and 3200 divisible by 7 but not a multiple of 5 (comma-separated, one line).

PYTHON

```
res = [str(i) for i in range(2000, 3201) if i % 7 == 0 and i % 5 != 0]
print(','.join(res[:12]), '...')
print('count =', len(res))
```

RESULT ON THE REAL CASE DATA

```
2002,2009,2016,2023,2037,2044,2051,2058,2072,2079,2086,2093 ...
count = 138
```

5 Count elements of each data type in [2, 3, 'Py', '10', 1, 'SQL', 5.5, True, 3, 'John', None, 7].

PYTHON

```
l1 = [2, 3, 'Py', '10', 1, 'SQL', 5.5, True, 3, 'John', None, 7]
from collections import Counter
print(dict(Counter(type(x).__name__ for x in l1)))
```

RESULT ON THE REAL CASE DATA

```
{'int': 5, 'str': 4, 'float': 1, 'bool': 1, 'NoneType': 1}
```

6 Add all numeric values in the list.

PYTHON

```python
l1 = [2, 3, 'Py', '10', 1, 'SQL', 5.5, True, 3, 'John', None, 7]
# bool is a subclass of int; exclude it explicitly
print(sum(x for x in l1 if isinstance(x, (int, float)) and not isinstance(x, bool)))
```

RESULT ON THE REAL CASE DATA

```
21.5
```

7 Concatenate all strings with a hyphen.

PYTHON

```python
l1 = [2, 3, 'Py', '10', 1, 'SQL', 5.5, True, 3, 'John', None, 7]
print('-'.join(x for x in l1 if isinstance(x, str)))
```

RESULT ON THE REAL CASE DATA

```
Py-10-SQL-John
```

8 UDF returning sum of numbers (excluding bool) and count of strings.

PYTHON

```python
def my_func(lst):
    s = sum(x for x in lst if isinstance(x, (int, float)) and not isinstance(x, bool))
    c = sum(isinstance(x, str) for x in lst)
    return {'Sum': s, 'Count_of_Strs': c}
print(my_func([2, 3, 'Py', '10', 1, 'SQL', 5.5, True, 3, 'John', None, 7]))
```

RESULT ON THE REAL CASE DATA

```
{'Sum': 21.5, 'Count_of_Strs': 4}
```

9 Get odd numbers from [5, 7, 22, 97, 54, 62, 77, 23, 73, 61] using (i) loop (ii) list comprehension (iii) lambda + filter.

PYTHON

```python
li = [5, 7, 22, 97, 54, 62, 77, 23, 73, 61]
odd = []
for x in li:
    if x % 2: odd.append(x)
print('loop      :', odd)
```

```
print('comprehension:', [x for x in li if x % 2])
print('filter    :', list(filter(lambda x: x % 2, li)))
```

RESULT ON THE REAL CASE DATA

```
loop      : [5, 7, 97, 77, 23, 73, 61]
comprehension: [5, 7, 97, 77, 23, 73, 61]
filter    : [5, 7, 97, 77, 23, 73, 61]
```

10 UDF returning descriptives [sum, count, min, mean, max] for n numbers.

PYTHON

```
def describe(*nums):
    return {'sum': sum(nums), 'count': len(nums), 'min': min(nums), 'mean':
sum(nums)/len(nums), 'max': max(nums)}
print(describe(4, 8, 15, 16, 23, 42))
```

RESULT ON THE REAL CASE DATA

```
{'sum': 108, 'count': 6, 'min': 4, 'mean': 18.0, 'max': 42}
```

11 UDF for area of square/rectangle/circle: shape positional, dimensions as kwargs, with validation.

PYTHON

```
import math
def area(shape, **kw):
    need = {'square': ['side'], 'rectangle': ['length', 'width'], 'circle': ['radius']}
    if shape not in need:
        return f'Unknown shape: {shape}'
    missing = [k for k in need[shape] if k not in kw]
    if missing:
        return f"Please enter {', '.join(repr(m) for m in missing)} for a {shape}"
    if shape == 'square': return kw['side'] ** 2
    if shape == 'rectangle': return kw['length'] * kw['width']
    return round(math.pi * kw['radius'] ** 2, 2)
print(area('square', side=4)); print(area('rectangle', length=5, width=3));
print(area('circle', radius=2)); print(area('square', length=4))
```

RESULT ON THE REAL CASE DATA

```
16
15
```

```
12.57
Please enter 'side' for a square
```

12 UDF to reconcile two month lists (matched, only in l1, only in l2).

PYTHON

```python
l1 = ['January', 'February', 'March', 'May', 'June', 'September', 'December']
l2 = ['January', 'February', 'April', 'June', 'October', 'December']
def func(a, b):
    return {'Matched': [x for x in a if x in b], 'Only in l1': [x for x in a if x not in
b], 'Only in l2': [x for x in b if x not in a]}
print(func(l1, l2))
```

RESULT ON THE REAL CASE DATA

```
{'Matched': ['January', 'February', 'June', 'December'], 'Only in l1': ['March', 'May',
'September'], 'Only in l2': ['April', 'October']}
```

13 UDF to check if a number is prime.

PYTHON

```python
def is_prime(n):
    if n < 2: return False
    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0: return False
    return True
print({n: is_prime(n) for n in [1, 2, 9, 17, 97, 100]})
```

RESULT ON THE REAL CASE DATA

```
{1: False, 2: True, 9: False, 17: True, 97: True, 100: False}
```

14 Factorial of given numbers (comma-separated output).

PYTHON

```python
def fact(n):
    return 1 if n <= 1 else n * fact(n - 1)
nums = '8'   # input()
print(','.join(str(fact(int(x))) for x in nums.split(',')))
```

RESULT ON THE REAL CASE DATA

```
40320
```

15 Dictionary {i: i*i} for 1..n.

PYTHON

```python
n = 8   # int(input())
print({i: i * i for i in range(1, n + 1)})
```

RESULT ON THE REAL CASE DATA

```
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25, 6: 36, 7: 49, 8: 64}
```

16 Comma-separated numbers → list and tuple.

PYTHON

```python
s = '34,67,55,33,12,98'   # input()
l = s.split(','); print(l, tuple(l))
```

RESULT ON THE REAL CASE DATA

```
['34', '67', '55', '33', '12', '98'] ('34', '67', '55', '33', '12', '98')
```

17 Sort comma-separated words alphabetically.

PYTHON

```python
s = 'without,hello,bag,world'
print(','.join(sorted(s.split(','))))
```

RESULT ON THE REAL CASE DATA

```
bag,hello,without,world
```

18 Remove duplicate words and sort.

PYTHON

```python
s = 'hello world and practice makes perfect and hello world again'
```

```
print(' '.join(sorted(set(s.split()))))
```

RESULT ON THE REAL CASE DATA

```
again and hello makes perfect practice world
```

19 Count upper-case and lower-case letters.

PYTHON

```
s = 'Hello world!'
print('UPPER CASE', sum(ch.isupper() for ch in s), 'LOWER CASE', sum(ch.islower() for ch
in s))
```

RESULT ON THE REAL CASE DATA

```
UPPER CASE 1 LOWER CASE 9
```

20 Reverse a string.

PYTHON

```
s = 'abcd123'
print(s[::-1])
```

RESULT ON THE REAL CASE DATA

```
321dcba
```

Python Exercise 2: Data Importing, Manipulation, Analysis and Visualisation (90 questions)

Ninety guided exercises in 8 sections: creating DataFrames, deleting data, filtering and sorting, aggregation, lambda/apply, joins, date-time operations and basic charts. Each answer below was run on the provided files.

Data

wine.txt, chipotle.tsv, occupation.csv, student-mat/por.csv, cars1/2.csv, wind.data, appl_1980_2014.csv, Online_Retail.csv, FMCG_Company_Data_2019.csv

Tool and module

Python (pandas, numpy, matplotlib)
Python for Data Science

1 Import the necessary libraries.

PYTHON

```
import pandas as pd, numpy as np, matplotlib.pyplot as plt, datetime as dt, re
print('imported')
```

RESULT ON THE REAL CASE DATA

```
imported
```

2–3 Create the dictionary and assign it to a DataFrame called pokemon.

PYTHON

```
raw_data = {'name': ['Bulbasaur', 'Charmander', 'Squirtle', 'Caterpie'], 'evolution':
['Ivysaur', 'Charmeleon', 'Wartortle', 'Metapod'],
            'type': ['grass', 'fire', 'water', 'bug'], 'hp': [45, 39, 44, 45], 'pokedex':
['yes', 'no', 'yes', 'no']}
pokemon = pd.DataFrame(raw_data); out = pokemon
```

RESULT ON THE REAL CASE DATA

	name	evolution	type	hp	pokedex
0	Bulbasaur	Ivysaur	grass	45	yes
1	Charmander	Charmeleon	fire	39	no
2	Squirtle	Wartortle	water	44	yes

	name	evolution	type	hp	pokedex
3	Caterpie	Metapod	bug	45	no

4 Reorder columns to name, type, hp, evolution, pokedex.

PYTHON

```
pokemon = pokemon[['name', 'type', 'hp', 'evolution', 'pokedex']]; out = pokemon
```

RESULT ON THE REAL CASE DATA

	name	type	hp	evolution	pokedex
0	Bulbasaur	grass	45	Ivysaur	yes
1	Charmander	fire	39	Charmeleon	no
2	Squirtle	water	44	Wartortle	yes
3	Caterpie	bug	45	Metapod	no

5 Add a column 'place'.

PYTHON

```
pokemon['place'] = ['park', 'street', 'lake', 'forest']; out = pokemon
```

RESULT ON THE REAL CASE DATA

	name	type	hp	evolution	pokedex	place
0	Bulbasaur	grass	45	Ivysaur	yes	park
1	Charmander	fire	39	Charmeleon	no	street
2	Squirtle	water	44	Wartortle	yes	lake
3	Caterpie	bug	45	Metapod	no	forest

6 Data type of each column.

PYTHON

```
out = pokemon.dtypes.astype(str)
```

RESULT ON THE REAL CASE DATA

	0
name	str
type	str
hp	int64
evolution	str
pokedex	str
place	str

7 Info of the DataFrame.

PYTHON

```
pokemon.info()
```

RESULT ON THE REAL CASE DATA

```
<class 'pandas.DataFrame'>
RangeIndex: 4 entries, 0 to 3
Data columns (total 6 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   name       4 non-null      str
 1   type       4 non-null      str
 2   hp         4 non-null      int64
 3   evolution  4 non-null      str
 4   pokedex    4 non-null      str
 5   place      4 non-null      str
dtypes: int64(1), str(5)
memory usage: 324.0 bytes
```

8 Import wine.txt without a header row.

PYTHON

```
wine = pd.read_csv('wine.txt', header=None); print(wine.shape)
```

RESULT ON THE REAL CASE DATA

```
(178, 14)
```

9 Delete the 1st, 4th, 7th, 9th, 11th, 13th and 14th columns.

PYTHON

```
wine = wine.drop(columns=wine.columns[[0, 3, 6, 8, 10, 12, 13]]); print(wine.shape)
```

RESULT ON THE REAL CASE DATA

```
(178, 7)
```

10 Name the columns alcohol, malic_acid, alcalinity_of_ash, magnesium, flavanoids, proanthocyanins, hue.

PYTHON

```
wine.columns = ['alcohol', 'malic_acid', 'alcalinity_of_ash', 'magnesium', 'flavanoids',
'proanthocyanins', 'hue']; out = wine.head()
```

RESULT ON THE REAL CASE DATA

	alcohol	malic_acid	alcalinity_of_ash	magnesium	flavanoids	proanthocyanins	hue
0	14.23	1.71	15.60	127	3.06	2.29	1.04
1	13.20	1.78	11.20	100	2.76	1.28	1.05
2	13.16	2.36	18.60	101	3.24	2.81	1.03
3	14.37	1.95	16.80	113	3.49	2.18	0.86
4	13.24	2.59	21.00	118	2.69	1.82	1.04

11 Set the first 3 values of alcohol to NaN.

PYTHON

```
wine.iloc[:3, 0] = np.nan; out = wine.head()
```

RESULT ON THE REAL CASE DATA

	alcohol	malic_acid	alcalinity_of_ash	magnesium	flavanoids	proanthocyanins	hue
0	NaN	1.71	15.60	127	3.06	2.29	1.04
1	NaN	1.78	11.20	100	2.76	1.28	1.05
2	NaN	2.36	18.60	101	3.24	2.81	1.03
3	14.37	1.95	16.80	113	3.49	2.18	0.86

	alcohol	malic_acid	alcalinity_of_ash	magnesium	flavanoids	proanthocyanins	hue
4	13.24	2.59	21.00	118	2.69	1.82	1.04

12 Set rows 3 and 4 of magnesium to NaN.

PYTHON

```
wine.loc[[2, 3], 'magnesium'] = np.nan; out = wine.head()
```

RESULT ON THE REAL CASE DATA

	alcohol	malic_acid	alcalinity_of_ash	magnesium	flavanoids	proanthocyanins	hue
0	NaN	1.71	15.60	127.00	3.06	2.29	1.04
1	NaN	1.78	11.20	100.00	2.76	1.28	1.05
2	NaN	2.36	18.60	NaN	3.24	2.81	1.03
3	14.37	1.95	16.80	NaN	3.49	2.18	0.86
4	13.24	2.59	21.00	118.00	2.69	1.82	1.04

13 Fill NaN with 10 in alcohol and 100 in magnesium.

PYTHON

```
wine = wine.fillna({'alcohol': 10, 'magnesium': 100}); out = wine.head()
```

RESULT ON THE REAL CASE DATA

	alcohol	malic_acid	alcalinity_of_ash	magnesium	flavanoids	proanthocyanins	hue
0	10.00	1.71	15.60	127.00	3.06	2.29	1.04
1	10.00	1.78	11.20	100.00	2.76	1.28	1.05
2	10.00	2.36	18.60	100.00	3.24	2.81	1.03
3	14.37	1.95	16.80	100.00	3.49	2.18	0.86
4	13.24	2.59	21.00	118.00	2.69	1.82	1.04

14 Count missing values in all columns.

PYTHON

```
out = wine.isna().sum()
```

RESULT ON THE REAL CASE DATA

	0
alcohol	0
malic_acid	0
alcalinity_of_ash	0
magnesium	0
flavanoids	0
proanthocyanins	0
hue	0

15 Array of 10 random numbers up to 10.

PYTHON

```
np.random.seed(1); rnd = np.random.randint(10, size=10); print(rnd)
```

RESULT ON THE REAL CASE DATA

```
[5 8 9 5 0 0 1 7 6 9]
```

16 Set rows at those random positions to NaN in alcohol.

PYTHON

```
wine.loc[rnd, 'alcohol'] = np.nan; print('done')
```

RESULT ON THE REAL CASE DATA

```
done
```

17 How many missing values now?

PYTHON

```
print(wine.isna().sum().sum())
```

RESULT ON THE REAL CASE DATA

```
7
```

18 Print only non-null alcohol values.

PYTHON

```
out = wine.loc[wine['alcohol'].notna(), 'alcohol']
```

RESULT ON THE REAL CASE DATA

	alcohol
2	10.00
3	14.37
4	13.24
10	14.10
11	14.12
12	13.75
13	14.75
14	14.38
15	13.63
16	14.30
17	13.83
18	14.19
19	13.64
20	14.06
21	12.93

Showing first 15 of 171 rows.

19 Delete rows with missing values.

PYTHON

```
wine = wine.dropna(); print(wine.shape)
```

RESULT ON THE REAL CASE DATA

```
(171, 7)
```

20 Reset the index.

PYTHON

```
wine = wine.reset_index(drop=True); out = wine.head()
```

RESULT ON THE REAL CASE DATA

	alcohol	malic_acid	alcalinity_of_ash	magnesium	flavanoids	proanthocyanins	hue
0	10.00	2.36	18.60	100.00	3.24	2.81	1.03
1	14.37	1.95	16.80	100.00	3.49	2.18	0.86
2	13.24	2.59	21.00	118.00	2.69	1.82	1.04
3	14.10	2.16	18.00	105.00	3.32	2.38	1.25
4	14.12	1.48	16.80	95.00	2.43	1.57	1.17

21 chipotle: how many products cost more than $10?

PYTHON

```
chipo = pd.read_csv('chipotle.tsv', sep='\t')
chipo['item_price'] = chipo['item_price'].str.replace('$', '', regex=False).astype(float)
print('Rows priced above $10:', (chipo['item_price'] > 10).sum())
u = chipo.drop_duplicates(['item_name', 'quantity'])
print('Distinct products (single quantity) above $10:', u[(u['quantity'] == 1) &
(u['item_price'] > 10)]['item_name'].nunique())
```

RESULT ON THE REAL CASE DATA

```
Rows priced above $10: 1130
Distinct products (single quantity) above $10: 12
```

22 Print the DataFrame and its info.

PYTHON

```
chipo.info(); out = chipo.head()
```

RESULT ON THE REAL CASE DATA

```
<class 'pandas.DataFrame'>
RangeIndex: 4622 entries, 0 to 4621
Data columns (total 5 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   order_id            4622 non-null   int64
 1   quantity            4622 non-null   int64
 2   item_name           4622 non-null   str
 3   choice_description  3376 non-null   str
 4   item_price          4622 non-null   float64
dtypes: float64(1), int64(2), str(2)
memory usage: 180.7 KB
```

	order_id	quantity	item_name	choice_description	item_price
0	1	1	Chips and Fresh Tomato Salsa	NaN	2.39
1	1	1	Izze	[Clementine]	3.39
2	1	1	Nantucket Nectar	[Apple]	3.39
3	1	1	Chips and Tomatillo-Green Chili Salsa	NaN	2.39
4	2	2	Chicken Bowl	[Tomatillo-Red Chili Salsa (Hot), [Black Beans, Rice, Cheese, Sour Cream]]	16.98

23 Price of each item: drop duplicates on item_name and quantity, keep item_name and item_price, sort high to low.

PYTHON

```python
p = chipo.drop_duplicates(['item_name', 'quantity'])
p = p[p['quantity'] == 1][['item_name', 'item_price']].sort_values('item_price',
ascending=False); out = p
```

RESULT ON THE REAL CASE DATA

	item_name	item_price
1132	Carnitas Salad Bowl	11.89
1229	Barbacoa Salad Bowl	11.89
606	Steak Salad Bowl	11.89
39	Barbacoa Bowl	11.75
7	Steak Burrito	11.75
168	Barbacoa Crispy Tacos	11.75

	item_name	item_price
57	Veggie Burrito	11.25
62	Veggie Bowl	11.25
186	Veggie Salad Bowl	11.25
738	Veggie Soft Tacos	11.25
250	Chicken Salad	10.98
5	Chicken Bowl	10.98
8	Steak Soft Tacos	9.25
92	Steak Crispy Tacos	9.25
554	Carnitas Crispy Tacos	9.25

Showing first 15 of 50 rows.

24 Sort by item name.

PYTHON

```
out = chipo.sort_values('item_name').head(10)
```

RESULT ON THE REAL CASE DATA

	order_id	quantity	item_name	choice_description	item_price
3389	1360	2	6 Pack Soft Drink	[Diet Coke]	12.98
341	148	1	6 Pack Soft Drink	[Diet Coke]	6.49
1849	749	1	6 Pack Soft Drink	[Coke]	6.49
1860	754	1	6 Pack Soft Drink	[Diet Coke]	6.49
2713	1076	1	6 Pack Soft Drink	[Coke]	6.49
3422	1373	1	6 Pack Soft Drink	[Coke]	6.49
553	230	1	6 Pack Soft Drink	[Diet Coke]	6.49
1916	774	1	6 Pack Soft Drink	[Diet Coke]	6.49
1922	776	1	6 Pack Soft Drink	[Coke]	6.49
1937	784	1	6 Pack Soft Drink	[Diet Coke]	6.49

25 Quantity of the most expensive item ordered.

PYTHON

```
out = chipo.sort_values('item_price', ascending=False).head(1)
```

RESULT ON THE REAL CASE DATA

	order_id	quantity	item_name	choice_description	item_price
3598	1443	15	Chips and Fresh Tomato Salsa	NaN	44.25

26 How many times was Veggie Salad Bowl ordered?

PYTHON

```
print(chipo[chipo['item_name'] == 'Veggie Salad Bowl']['quantity'].sum())
```

RESULT ON THE REAL CASE DATA

```
18
```

27 How many times did people order more than one Canned Soda?

PYTHON

```
print(len(chipo[(chipo['item_name'] == 'Canned Soda') & (chipo['quantity'] > 1)]))
```

RESULT ON THE REAL CASE DATA

```
20
```

28 Import occupation.csv as users.

PYTHON

```
users = pd.read_csv('occupation.csv', sep='|', index_col='user_id'); out = users.head()
```

RESULT ON THE REAL CASE DATA

user_id	age	gender	occupation	zip_code
1	24	M	technician	85711
2	53	F	other	94043

user_id	age	gender	occupation	zip_code
3	23	M	writer	32067
4	24	M	technician	43537
5	33	F	other	15213

29 Mean age per occupation.

PYTHON

```
out = users.groupby('occupation')['age'].mean().round(1)
```

RESULT ON THE REAL CASE DATA

occupation	age
administrator	38.70
artist	31.40
doctor	43.60
educator	42.00
engineer	36.40
entertainment	29.20
executive	38.70
healthcare	41.60
homemaker	32.60
lawyer	36.80
librarian	40.00
marketing	37.60
none	26.60
other	34.50
programmer	33.10

Showing first 15 of 21 rows.

30 Male ratio per occupation, sorted high to low (use np.where).

PYTHON

```
users['male'] = np.where(users['gender'] == 'M', 1, 0)
out = (users.groupby('occupation')['male'].mean() *
100).round(1).sort_values(ascending=False)
```

RESULT ON THE REAL CASE DATA

occupation	male
doctor	100.00
engineer	97.00
technician	96.30
retired	92.90
programmer	90.90
executive	90.60
scientist	90.30
entertainment	88.90
lawyer	83.30
salesman	75.00
educator	72.60
student	69.40
other	65.70
marketing	61.50
writer	57.80

Showing first 15 of 21 rows.

31 Min and max age per occupation.

PYTHON

```
out = users.groupby('occupation')['age'].agg(['min', 'max'])
```

RESULT ON THE REAL CASE DATA

occupation	min	max
administrator	21	70
artist	19	48
doctor	28	64
educator	23	63
engineer	22	70
entertainment	15	50
executive	22	69
healthcare	22	62
homemaker	20	50
lawyer	21	53
librarian	23	69
marketing	24	55
none	11	55
other	13	64
programmer	20	63

Showing first 15 of 21 rows.

32 Mean age per occupation and gender.

PYTHON

```
out = users.groupby(['occupation', 'gender'])['age'].mean().round(1).unstack()
```

RESULT ON THE REAL CASE DATA

occupation \ gender	F	M
administrator	40.60	37.20
artist	30.30	32.30
doctor	NaN	43.60
educator	39.10	43.10

gender	F	M
occupation		
engineer	29.50	36.60
entertainment	31.00	29.00
executive	44.00	38.20
healthcare	39.80	45.40
homemaker	34.20	23.00
lawyer	39.50	36.20
librarian	40.00	40.00
marketing	37.20	37.90
none	36.50	18.60
other	35.50	34.00
programmer	32.20	33.20

Showing first 15 of 21 rows.

33 Percentage of women and men per occupation.

PYTHON

```
out = (pd.crosstab(users['occupation'], users['gender'], normalize='index') *
100).round(1)
```

RESULT ON THE REAL CASE DATA

gender	F	M
occupation		
administrator	45.60	54.40
artist	46.40	53.60
doctor	0.00	100.00
educator	27.40	72.60
engineer	3.00	97.00
entertainment	11.10	88.90
executive	9.40	90.60
healthcare	68.80	31.20

gender	F	M
occupation		
homemaker	85.70	14.30
lawyer	16.70	83.30
librarian	56.90	43.10
marketing	38.50	61.50
none	44.40	55.60
other	34.30	65.70
programmer	9.10	90.90

Showing first 15 of 21 rows.

34 Import student-mat and student-por and append them as df.

PYTHON

```
df = pd.concat([pd.read_csv('student-mat.csv'), pd.read_csv('student-por.csv')],
ignore_index=True); print(df.shape)
```

RESULT ON THE REAL CASE DATA

```
(1044, 33)
```

35 Slice columns from school to guardian.

PYTHON

```
stud = df.loc[:, 'school':'guardian']; print(stud.shape)
```

RESULT ON THE REAL CASE DATA

```
(1044, 12)
```

36 Lambda function that capitalises a string.

PYTHON

```
capitalizer = lambda x: x.capitalize()
```

```
print(capitalizer('at_home'))
```

RESULT ON THE REAL CASE DATA

```
At_home
```

37 Capitalise Mjob and Fjob.

PYTHON

```
print(stud['Mjob'].apply(capitalizer).head(3).tolist())
```

RESULT ON THE REAL CASE DATA

```
['At_home', 'At_home', 'At_home']
```

38 Print the last rows.

PYTHON

```
out = stud.tail()
```

RESULT ON THE REAL CASE DATA

	school	sex	age	address	famsize	Pstatus	Medu	Fedu	Mjob	Fjob	re
1039	MS	F	19	R	GT3	T	2	3	services	other	co
1040	MS	F	18	U	LE3	T	3	1	teacher	services	co
1041	MS	F	18	U	GT3	T	1	1	other	other	co
1042	MS	M	17	U	LE3	T	3	1	services	services	co
1043	MS	M	18	R	LE3	T	3	2	services	other	co

39 Why is the original still lowercase? Fix it.

PYTHON

```
# apply() returns a NEW Series; it must be assigned back
stud['Mjob'] = stud['Mjob'].apply(capitalizer)
stud['Fjob'] = stud['Fjob'].apply(capitalizer)
out = stud[['Mjob', 'Fjob']].tail()
```

RESULT ON THE REAL CASE DATA

	Mjob	Fjob
1039	Services	Other
1040	Teacher	Services
1041	Other	Other
1042	Services	Services
1043	Services	Other

40 Function majority → boolean column legal_drinker (age > 17).

PYTHON

```
def majority(age):
    return age > 17
stud['legal_drinker'] = stud['age'].apply(majority); out = stud[['age',
'legal_drinker']].head()
```

RESULT ON THE REAL CASE DATA

	age	legal_drinker
0	18	True
1	17	False
2	15	False
3	15	False
4	16	False

41 Multiply every number in the dataset by 10.

PYTHON

```
num = stud.select_dtypes('number').columns
out = stud[num].mul(10).head()
```

RESULT ON THE REAL CASE DATA

	age	Medu	Fedu
0	180	40	40
1	170	10	10
2	150	10	10

	age	Medu	Fedu
3	150	40	20
4	160	30	30

42 Import cars1 and cars2.

PYTHON

```
cars1 = pd.read_csv('cars1.csv'); cars2 = pd.read_csv('cars2.csv'); print(cars1.shape, cars2.shape)
```

RESULT ON THE REAL CASE DATA

```
(198, 14) (200, 9)
```

43 Inspect cars1 (type, head, tail, columns, info, dtypes, shape, describe, value_counts...).

PYTHON

```
print(type(cars1)); print(cars1.columns.tolist()); print(cars1.shape, cars1.size, cars1.ndim)
out = cars1.describe().round(2)
```

RESULT ON THE REAL CASE DATA

```
<class 'pandas.DataFrame'>
['mpg', 'cylinders', 'displacement', 'horsepower', 'weight', 'acceleration', 'model', 'origin', 'car', 'Unnamed: 9', 'Unnamed: 10', 'Unnamed: 11', 'Unnamed: 12', 'Unnamed: 13']
(198, 14) 2772 2
```

	mpg	cylinders	displacement	weight	acceleration	model	origin	Unnamed: 9
count	198.00	198.00	198.00	198.00	198.00	198.00	198.00	0.00
mean	19.72	5.90	223.47	3,177.89	15.01	72.82	1.44	NaN
std	5.81	1.79	115.18	934.78	2.87	1.87	0.71	NaN
min	9.00	3.00	68.00	1,613.00	8.00	70.00	1.00	NaN
25%	15.00	4.00	113.25	2,302.50	13.00	71.00	1.00	NaN
50%	19.00	6.00	228.00	3,030.00	15.00	73.00	1.00	NaN

	mpg	cylinders	displacement	weight	acceleration	model	origin	Unnamed: 9
75%	24.38	8.00	318.00	4,080.75	16.80	74.00	2.00	NaN
max	35.00	8.00	455.00	5,140.00	23.50	76.00	3.00	NaN

44 Remove the unnamed blank columns from cars1.

PYTHON

```
cars1 = cars1.loc[:, 'mpg':'car']; print(cars1.columns.tolist())
```

RESULT ON THE REAL CASE DATA

```
['mpg', 'cylinders', 'displacement', 'horsepower', 'weight', 'acceleration', 'model',
'origin', 'car']
```

45 Number of observations in each dataset.

PYTHON

```
print('cars1:', len(cars1), '| cars2:', len(cars2))
```

RESULT ON THE REAL CASE DATA

```
cars1: 198 | cars2: 200
```

46 Join cars1 and cars2 into cars.

PYTHON

```
cars = pd.concat([cars1, cars2], ignore_index=True); print(cars.shape)
```

RESULT ON THE REAL CASE DATA

```
(398, 9)
```

47 Random 'owners' Series between 15,000 and 73,000.

PYTHON

```
np.random.seed(7); owners = np.random.randint(15000, 73001, size=len(cars));
print(owners[:5])
```

RESULT ON THE REAL CASE DATA

```
[64689 25742 53467 68459 15919]
```

48 Add owners to cars.

PYTHON

```
cars['owners'] = owners; out = cars.head()
```

RESULT ON THE REAL CASE DATA

	mpg	cylinders	displacement	horsepower	weight	acceleration	model	origin	car
0	18.00	8	307	130	3504	12.00	70	1	chev chev malil
1	15.00	8	350	165	3693	11.50	70	1	buicl skyla 320
2	18.00	8	318	150	3436	11.00	70	1	plym satel
3	16.00	8	304	150	3433	12.00	70	1	amc rebe
4	17.00	8	302	140	3449	10.50	70	1	ford torin

49 Current date-time, year, month, week number, weekday, day of year, day of month, day of week.

PYTHON

```
now = dt.datetime(2026, 10, 27, 10, 30)   # use dt.datetime.now() in practice
print(now, now.year, now.month, now.isocalendar()[1], now.strftime('%A'),
now.timetuple().tm_yday, now.day, now.weekday())
```

RESULT ON THE REAL CASE DATA

```
2026-10-27 10:30:00 2026 10 44 Tuesday 300 27 1
```

50 Convert 'Jul 1 2014 2:43PM' to datetime.

PYTHON

```
print(dt.datetime.strptime('Jul 1 2014 2:43PM', '%b %d %Y %I:%M%p'))
```

RESULT ON THE REAL CASE DATA

```
2014-07-01 14:43:00
```

51 Subtract five days from 2015-06-22.

PYTHON

```
print(dt.date(2015, 6, 22) - dt.timedelta(days=5))
```

RESULT ON THE REAL CASE DATA

```
2015-06-17
```

52 Unix timestamp 1284105682 to readable date.

PYTHON

```
print(dt.datetime.fromtimestamp(1284105682, dt.timezone.utc).strftime('%Y-%m-%d
%H:%M:%S'))
```

RESULT ON THE REAL CASE DATA

```
2010-09-10 08:01:22
```

53 Convert ['07Sep59','01Jan55','15Dec47','11Jul42'] to dates in 19XX.

PYTHON

```
DoB = pd.Series(['07Sep59', '01Jan55', '15Dec47', '11Jul42'])
d = pd.to_datetime(DoB, format='%d%b%y')
d = d.where(d.dt.year < 2000, d - pd.DateOffset(years=100)); out = d
```

RESULT ON THE REAL CASE DATA

	0
0	1959-09-07

	0
1	1955-01-01
2	1947-12-15
3	1942-07-11

54 Days between two dates.

PYTHON

```
print((dt.date(2026, 10, 27) - dt.date(2026, 1, 1)).days, 'days')
```

RESULT ON THE REAL CASE DATA

```
299 days
```

55 Convert '15Dec1989' and display as 'Friday, 15 Dec 89'.

PYTHON

```
d = pd.to_datetime(pd.Series(['15Dec1989']), format='%d%b%Y')
print(d.dt.strftime('%A, %d %b %y')[0])
```

RESULT ON THE REAL CASE DATA

```
Friday, 15 Dec 89
```

56–58 Import wind.data, build a proper date index, fix years < 70 to 19XX, dtype datetime64.

PYTHON

```
data = pd.read_csv('wind.data', sep=r'\s+|,', engine='python')
data['Date'] = pd.to_datetime(dict(year=data.Yr + 1900, month=data.Mo, day=data.Dy))
data = data.drop(columns=['Yr', 'Mo', 'Dy']).set_index('Date'); print(data.index.dtype);
out = data.head()
```

RESULT ON THE REAL CASE DATA

```
datetime64[us]
```

Date	RPT	VAL	ROS	KIL	SHA	BIR	DUB	CLA	MUL	CLO	BEL	MAL
1961-01-01	15.04	14.96	13.17	9.29	NaN	9.87	13.67	10.25	10.83	12.58	18.50	15.04
1961-01-02	14.71	NaN	10.83	6.50	12.62	7.67	11.50	10.04	9.79	9.67	17.54	13.83
1961-01-03	18.50	16.88	12.33	10.13	11.17	6.17	11.25	NaN	8.50	7.67	12.75	12.71
1961-01-04	10.58	6.63	11.75	4.58	4.54	2.88	8.63	1.79	5.83	5.88	5.46	10.88
1961-01-05	13.33	13.25	11.42	6.17	10.71	8.21	11.92	6.54	10.92	10.34	12.92	11.83

59 Missing values per location.

PYTHON

```
out = data.isna().sum()
```

RESULT ON THE REAL CASE DATA

	0
RPT	6
VAL	3
ROS	2
KIL	5
SHA	2
BIR	0
DUB	3
CLA	2
MUL	3
CLO	1
BEL	0
MAL	4

60 Total non-missing values.

PYTHON

```
print(data.notna().sum().sum())
```

RESULT ON THE REAL CASE DATA

```
78857
```

61 Mean wind speed over all locations and times.

PYTHON

```
print(round(data.stack().mean(), 4))
```

RESULT ON THE REAL CASE DATA

```
10.2279
```

62 loc_stats: min, max, mean, std per location.

PYTHON

```
loc_stats = data.agg(['min', 'max', 'mean', 'std']).T; out = loc_stats.round(2)
```

RESULT ON THE REAL CASE DATA

	min	max	mean	std
RPT	0.67	35.80	12.36	5.62
VAL	0.21	33.37	10.64	5.27
ROS	1.50	33.84	11.66	5.01
KIL	0.00	28.46	6.31	3.61
SHA	0.13	37.54	10.46	4.94
BIR	0.00	26.16	7.09	3.97
DUB	0.00	30.37	9.80	4.98
CLA	0.00	31.08	8.50	4.50
MUL	0.00	25.88	8.49	4.17
CLO	0.04	28.21	8.71	4.50

	min	max	mean	std
BEL	0.13	42.38	13.12	5.84
MAL	0.67	42.54	15.60	6.70

63 day_stats: min, max, mean, std across locations per day.

PYTHON

```
day_stats = data.agg(['min', 'max', 'mean', 'std'], axis=1); out =
day_stats.head().round(2)
```

RESULT ON THE REAL CASE DATA

Date	min	max	mean	std
1961-01-01	9.29	18.50	13.02	2.81
1961-01-02	6.50	17.54	11.34	3.19
1961-01-03	6.17	18.50	11.64	3.68
1961-01-04	1.79	11.75	6.62	3.20
1961-01-05	6.17	13.33	10.63	2.45

64 Average January wind speed per location (all years).

PYTHON

```
out = data[data.index.month == 1].mean().round(2)
```

RESULT ON THE REAL CASE DATA

	0
RPT	14.85
VAL	12.91
ROS	13.30
KIL	7.20
SHA	11.67
BIR	8.05
DUB	11.82

	0
CLA	9.51
MUL	9.54
CLO	10.05
BEL	14.55
MAL	18.03

65 Mean wind speed for each month (Jan 1961 and Jan 1962 different).

PYTHON

```
out = data.groupby(data.index.to_period('M')).mean().round(2).head()
```

RESULT ON THE REAL CASE DATA

Date	RPT	VAL	ROS	KIL	SHA	BIR	DUB	CLA	MUL	CLO	BEL	MAL
1961-01	14.84	11.99	13.43	7.74	11.07	8.59	11.18	9.25	9.09	10.11	13.88	14.70
1961-02	16.27	14.98	14.44	9.23	13.85	10.94	11.89	11.85	11.82	12.71	18.58	15.41
1961-03	10.89	11.30	10.75	7.28	10.51	8.87	9.64	9.83	10.29	11.25	16.41	15.72
1961-04	10.72	9.43	10.00	5.83	8.44	6.50	6.93	7.09	7.34	7.24	11.15	10.28
1961-05	9.86	8.85	10.82	5.91	9.49	6.57	7.60	8.18	8.04	8.50	11.90	12.01

66 Weekly min/max/mean/std across locations for the first 52 weeks (weeks start 2 Jan 1961).

PYTHON

```
w = data.loc['1961-01-02':].resample('W-SUN').agg(['min', 'max', 'mean',
'std']).iloc[:52]; print(w.shape); out = w.iloc[:5, :8].round(2)
```

RESULT ON THE REAL CASE DATA

```
(52, 48)
```

	RPT				VAL			
	min	max	mean	std	min	max	mean	std
Date								
1961-01-08	10.58	18.50	13.54	2.63	6.63	16.88	11.49	3.95
1961-01-15	9.04	19.75	12.47	3.56	3.54	12.08	8.97	3.15
1961-01-22	4.92	19.83	13.20	5.34	3.42	14.37	9.86	3.84
1961-01-29	13.62	25.04	19.88	4.62	9.96	23.91	16.14	5.17
1961-02-05	10.58	24.21	16.83	5.25	9.46	24.21	15.46	5.19

67 Import appl_1980_2014.csv as apple.

PYTHON

```
apple = pd.read_csv('appl_1980_2014.csv'); out = apple.head()
```

RESULT ON THE REAL CASE DATA

	Date	Open	High	Low	Close	Volume	Adj Close
0	2014-07-08	96.27	96.80	93.92	95.35	65130000	95.35
1	2014-07-07	94.14	95.99	94.10	95.97	56305400	95.97
2	2014-07-03	93.67	94.10	93.20	94.03	22891800	94.03
3	2014-07-02	93.87	94.06	93.09	93.48	28420900	93.48
4	2014-07-01	93.52	94.07	93.13	93.52	38170200	93.52

68 Column types.

PYTHON

```
out = apple.dtypes.astype(str)
```

RESULT ON THE REAL CASE DATA

	0
Date	str
Open	float64
High	float64

	0
Low	float64
Close	float64
Volume	int64
Adj Close	float64

69–70 Convert Date to datetime and set as index.

PYTHON

```
apple['Date'] = pd.to_datetime(apple['Date']); apple = apple.set_index('Date');
print(apple.index.dtype)
```

RESULT ON THE REAL CASE DATA

```
datetime64[us]
```

71 Any duplicate dates?

PYTHON

```
print('Duplicates:', apple.index.duplicated().any())
```

RESULT ON THE REAL CASE DATA

```
Duplicates: False
```

72 Sort so the oldest date is first.

PYTHON

```
apple = apple.sort_index(); out = apple.head(3)
```

RESULT ON THE REAL CASE DATA

Date	Open	High	Low	Close	Volume	Adj Close
1980-12-12	28.75	28.87	28.75	28.75	117258400	0.45
1980-12-15	27.38	27.38	27.25	27.25	43971200	0.42

Date	Open	High	Low	Close	Volume	Adj Close
1980-12-16	25.37	25.37	25.25	25.25	26432000	0.39

73 Last business day of each month.

PYTHON

```
out = apple.groupby(apple.index.to_period('M')).tail(1).head()
```

RESULT ON THE REAL CASE DATA

Date	Open	High	Low	Close	Volume	Adj Close
1980-12-31	34.25	34.25	34.13	34.13	8937600	0.53
1981-01-30	28.50	28.50	28.25	28.25	11547200	0.44
1981-02-27	26.50	26.75	26.50	26.50	3690400	0.41
1981-03-31	24.75	24.75	24.50	24.50	3998400	0.38
1981-04-30	28.38	28.62	28.38	28.38	3152800	0.44

74 Days between the first and last date.

PYTHON

```
print((apple.index.max() - apple.index.min()).days, 'days')
```

RESULT ON THE REAL CASE DATA

```
12261 days
```

75 How many months of data?

PYTHON

```
print(apple.index.to_period('M').nunique(), 'months')
```

RESULT ON THE REAL CASE DATA

```
404 months
```

76 Plot Adj Close with figure size 13.5 × 9 inches.

PYTHON

```
ax = apple['Adj Close'].plot(figsize=(13.5, 9), title='Apple adjusted close, 1980–2014')
```

RESULT ON THE REAL CASE DATA

77 Import Online_Retail.csv as online_rt.

PYTHON

```
online_rt = pd.read_csv('Online_Retail.csv', encoding='latin1'); print(online_rt.shape)
```

RESULT ON THE REAL CASE DATA

```
(541909, 8)
```

78–79 Bar chart of the top 10 countries by Quantity, excluding UK and negative quantities.

PYTHON

```
o = online_rt[online_rt['Quantity'] > 0]
top = o[o['Country'] != 'United Kingdom'].groupby('Country')
['Quantity'].sum().nlargest(10)
top.plot.bar(title='Top 10 countries by quantity (excl. UK)', rot=45); out = top
```

RESULT ON THE REAL CASE DATA

Country	Quantity
Netherlands	200937
EIRE	147447
Germany	119263
France	112104

Country	Quantity
Australia	84209
Sweden	36083
Switzerland	30630
Spain	27951
Japan	26016
Belgium	23237

80 Scatter of Quantity vs UnitPrice per CustomerID for the top 3 countries by revenue.

PYTHON

```
o = o.assign(Revenue=o['Quantity'] * o['UnitPrice'])
top3 = o.groupby('Country')['Revenue'].sum().nlargest(3).index
fig, ax = plt.subplots(1, 3, figsize=(13, 3.6))
for a, ctry in zip(ax, top3):
    g = o[o['Country'] == ctry].groupby('CustomerID')[['Quantity', 'UnitPrice']].sum()
    a.scatter(g['Quantity'], g['UnitPrice'], s=10); a.set_title(ctry);
a.set_xlabel('Quantity'); a.set_ylabel('UnitPrice')
```

RESULT ON THE REAL CASE DATA

81 Import FMCG_Company_Data_2019.csv as company_data.

PYTHON

```
company_data = pd.read_csv('FMCG_Company_Data_2019.csv'); out = company_data
```

RESULT ON THE REAL CASE DATA

	Month	FaceCream	FaceWash	ToothPaste	Soap	Shampo	Moisturizer	Total_Units
0	Jan-19	2500	1500	5200	9200	1200	1500	21100
1	Feb-19	2630	1200	5100	6100	2100	1200	18330

	Month	FaceCream	FaceWash	ToothPaste	Soap	Shampo	Moisturizer	Total_Units
2	Mar-19	2140	1340	4550	9550	3550	1340	22470
3	Apr-19	3400	1130	5870	8870	1870	1130	22270
4	May-19	3600	1740	4560	7760	1560	1740	20960
5	Jun-19	2760	1555	4890	7490	1890	1555	20140
6	Jul-19	2980	1120	4780	8980	1780	1120	29550
7	Aug-19	3700	1400	5860	9960	2860	1400	36140
8	Sep-19	3540	1780	6100	8100	2100	1780	23400
9	Oct-19	1990	1890	8300	10300	2300	1890	26670
10	Nov-19	2340	2100	7300	13300	2400	2100	41280
11	Dec-19	2900	1760	7400	14400	1800	1760	30020

82 Line chart of Total Revenue by month.

PYTHON

```
plt.plot(company_data['Month'], company_data['Total_Revenue'], marker='o');
plt.xlabel('Month'); plt.ylabel('Total Revenue'); plt.xticks(rotation=45)
```

RESULT ON THE REAL CASE DATA

83 Line chart of Total Units: dotted red line, legend lower right.

PYTHON

```
plt.plot(company_data['Month'], company_data['Total_Units'], 'r:', marker='o',
label='Total Units'); plt.xlabel('Month'); plt.ylabel('Total Units');
plt.legend(loc='lower right'); plt.xticks(rotation=45)
```

RESULT ON THE REAL CASE DATA

84 Multi-line plot of units per product per month.

PYTHON

```
prods = ['FaceCream', 'FaceWash', 'ToothPaste', 'Soap', 'Shampo', 'Moisturizer']
company_data.plot(x='Month', y=prods, marker='o', figsize=(10, 4), title='Units sold per
month by product')
```

RESULT ON THE REAL CASE DATA

85 Bar chart for Soap (and save it).

PYTHON

```
company_data.plot.bar(x='Month', y='Soap', rot=45, title='Soap units by month',
legend=False)
plt.savefig('/tmp/soap_bar.png')
```

RESULT ON THE REAL CASE DATA

86 Grouped bar chart for Soap, Shampo, ToothPaste per month.

PYTHON

```
company_data.plot.bar(x='Month', y=['Soap', 'Shampo', 'ToothPaste'], rot=45, figsize=(10,
4))
```

RESULT ON THE REAL CASE DATA

87 Histogram of Total Revenue.

PYTHON

```
company_data['Total_Revenue'].plot.hist(bins=6, title='Total revenue distribution')
```

RESULT ON THE REAL CASE DATA

88 Pie chart of total 2019 units by product.

PYTHON

```
s = company_data[prods].sum(); s.plot.pie(autopct='%1.1f%%', ylabel='', figsize=(5.5, 5.5)); out = s
```

RESULT ON THE REAL CASE DATA

	0
FaceCream	34480
FaceWash	18515
ToothPaste	69910
Soap	114010
Shampo	25410
Moisturizer	18515

89 Soap and FaceWash in subplots.

PYTHON

```
fig, ax = plt.subplots(2, 1, figsize=(9, 5), sharex=True)
ax[0].plot(company_data['Month'], company_data['Soap'], 'b-o'); ax[0].set_title('Soap')
ax[1].plot(company_data['Month'], company_data['FaceWash'], 'g-o');
ax[1].set_title('FaceWash'); plt.xticks(rotation=45)
```

RESULT ON THE REAL CASE DATA

90 Box plot of Total Profit.

PYTHON

```
company_data['Total_Profit'].plot.box(title='Total profit')
```

RESULT ON THE REAL CASE DATA

PYTHON FOR DATA SCIENCE · RESOLUTION

Python Case Study 1: Customer Analysis for Retail

A retail store wants to analyse day-to-day transactions and keep track of its customers across locations, including their purchases and returns across product categories. Build a report with the calculated metrics, summaries and inferences listed below.

Data

Customer.csv (5,647 customers: id, DOB, gender, city code) · Transactions.csv (23,053 rows: qty, rate, tax, total amount, store type; negative values are returns) · prod_cat_info.csv (category and sub-category codes).

Tool and module

Python (pandas, matplotlib)
Python for Data Science

1 Merge Customers, Product Hierarchy and Transactions as Customer_Final, keeping every customer who has transacted.

PYTHON

```
tran['tran_date'] = pd.to_datetime(tran['tran_date'].str.replace('/', '-'), dayfirst=True,
format='mixed')
cust['DOB'] = pd.to_datetime(cust['DOB'], dayfirst=True)
# transactions -> customers (left join keeps every transacting customer)
t = tran.merge(cust, how='left', left_on='cust_id', right_on='customer_Id')
# product table is keyed on BOTH category and sub-category code
Customer_Final = t.merge(prod, how='left', left_on=['prod_cat_code', 'prod_subcat_code'],
                         right_on=['prod_cat_code', 'prod_sub_cat_code'])
print('Shape of Customer_Final:', Customer_Final.shape)
print('Transactions without a customer match:',
Customer_Final['customer_Id'].isna().sum())
out = Customer_Final.head()
```

RESULT ON THE REAL CASE DATA

```
Shape of Customer_Final: (23053, 17)
Transactions without a customer match: 0
```

	transaction_id	cust_id	tran_date	prod_subcat_code	prod_cat_code	Qty	Rate	Tax
0	80712190438	270351	2014-02-28	1	1	-5	-772	405.3

	transaction_id	cust_id	tran_date	prod_subcat_code	prod_cat_code	Qty	Rate	Tax
1	29258453508	270384	2014-02-27	5	3	-5	-1497	785.9
2	51750724947	273420	2014-02-24	6	5	-2	-791	166.11
3	93274880719	271509	2014-02-24	11	6	-3	-1363	429.3
4	51750724947	273420	2014-02-23	6	5	-2	-791	166.11

Insight The product table must be joined on both **prod_cat_code** and **prod_sub_cat_code**; joining on the sub-category code alone duplicates rows because sub-category codes repeat across categories.

2a Summary report: column names and their data types.

PYTHON

```
out = Customer_Final.dtypes.astype(str).rename('dtype')
```

RESULT ON THE REAL CASE DATA

	dtype
transaction_id	int64
cust_id	int64
tran_date	datetime64[us]
prod_subcat_code	int64
prod_cat_code	int64
Qty	int64
Rate	int64
Tax	float64
total_amt	float64
Store_type	str
customer_Id	int64

	dtype
DOB	datetime64[us]
Gender	str
city_code	float64
prod_cat	str

Showing first 15 of 17 rows.

2b Summary report: top and bottom 10 observations.

PYTHON

```
print('TOP 10'); print(Customer_Final.head(10).iloc[:, :8].to_string())
print('\nBOTTOM 10'); print(Customer_Final.tail(10).iloc[:, :8].to_string())
```

RESULT ON THE REAL CASE DATA

```
TOP 10
   transaction_id  cust_id  tran_date  prod_subcat_code  prod_cat_code  Qty  Rate      Tax
0     80712190438   270351 2014-02-28                 1              1   -5  -772  405.300
1     29258453508   270384 2014-02-27                 5              3   -5 -1497  785.925
2     51750724947   273420 2014-02-24                 6              5   -2  -791  166.110
3     93274880719   271509 2014-02-24                11              6   -3 -1363  429.345
4     51750724947   273420 2014-02-23                 6              5   -2  -791  166.110
5     97439039119   272357 2014-02-23                 8              3   -2  -824  173.040
6     45649838090   273667 2014-02-22                11              6   -1 -1450  152.250
7     22643667930   271489 2014-02-22                12              6   -1 -1225  128.625
8     79792372943   275108 2014-02-22                 3              1   -3  -908  286.020
9     50076728598   269014 2014-02-21                 8              3   -4  -581  244.020

BOTTOM 10
       transaction_id  cust_id  tran_date  prod_subcat_code  prod_cat_code  Qty  Rate
Tax
23043     49882891062   271982 2011-01-25                10              5    4  1330
558.600
23044     14787475597   273982 2011-01-25                 4              3    5   969
508.725
23045     50691119572   273031 2011-01-25                 6              5    1  1148
120.540
23046     40893803228   272049 2011-01-25                11              6    3  1077
339.255
23047     30856003613   266866 2011-01-25                 4              2    2   444
93.240
23048     94340757522   274550 2011-01-25                12              5    1  1264
132.720
23049     89780862956   270022 2011-01-25                 4              1    1   677
71.085
23050     85115299378   271020 2011-01-25                 2              6    4  1052
441.840
23051     72870271171   270911 2011-01-25                11              5    3  1142
```

```
359.730
23052     77960931771   271961 2011-01-25               11             5    1   447
46.935
```

2c Five-number summary (min, Q1, median, Q3, max) for continuous variables.

PYTHON

```python
num = ['Qty', 'Rate', 'Tax', 'total_amt']
out = Customer_Final[num].quantile([0, .25, .5, .75, 1]).T
out.columns = ['Min', 'Q1', 'Median', 'Q3', 'Max']
```

RESULT ON THE REAL CASE DATA

	Min	Q1	Median	Q3	Max
Qty	-5.00	1.00	3.00	4.00	5.00
Rate	-1,499.00	312.00	710.00	1,109.00	1,500.00
Tax	7.35	98.28	199.08	365.71	787.50
total_amt	-8,270.92	762.45	1,754.74	3,569.15	8,287.50

2d Frequency tables for all categorical variables.

PYTHON

```python
for col in ['Store_type', 'Gender', 'city_code', 'prod_cat', 'prod_subcat']:
    print(f'--- {col}');
print(Customer_Final[col].value_counts(dropna=False).to_string()); print()
```

RESULT ON THE REAL CASE DATA

```
--- Store_type
Store_type
e-Shop            9311
MBR               4661
Flagship store    4577
TeleShop          4504

--- Gender
Gender
M      11811
F      11233
NaN        9

--- city_code
city_code
4.0     2422
```

```
3.0     2411
5.0     2360
7.0     2356
10.0    2333
8.0     2330
2.0     2270
1.0     2258
9.0     2178
6.0     2127
NaN        8

--- prod_cat
prod_cat
Books               6069
Electronics         4898
Home and kitchen    4129
Footwear            2999
Clothing            2960
Bags                1998

--- prod_subcat
prod_subcat
Women                  3048
Mens                   2912
Kids                   1997
Tools                  1062
Fiction                1043
Kitchen                1037
Children               1035
Comics                 1031
Mobiles                1031
Bath                   1023
Furnishing             1007
Non-Fiction            1004
DIY                     989
Cameras                 985
Personal Appliances     972
Academic                967
Computers               958
Audio and video         952
```

3 Histograms for all continuous variables and frequency bars for categorical variables.

PYTHON

```python
fig, ax = plt.subplots(1, 4, figsize=(12, 3))
for a, col in zip(ax, ['Qty', 'Rate', 'Tax', 'total_amt']):
    a.hist(Customer_Final[col], bins=30, color='#1F3864'); a.set_title(col)
fig, ax = plt.subplots(1, 3, figsize=(12, 3.2))
for a, col in zip(ax, ['Store_type', 'Gender', 'prod_cat']):
    Customer_Final[col].value_counts().plot.bar(ax=a, color='#E9A23B'); a.set_title(col);
a.tick_params(axis='x', rotation=30)
```

RESULT ON THE REAL CASE DATA

4a Time period of the available transaction data.

PYTHON

```
mn, mx = Customer_Final['tran_date'].min(), Customer_Final['tran_date'].max()
print(f'Answer: from {mn:%d %b %Y} to {mx:%d %b %Y} = {(mx-mn).days} days (~{(mx-
mn).days/30.44:.0f} months)')
```

RESULT ON THE REAL CASE DATA

```
Answer: from 25 Jan 2011 to 28 Feb 2014 = 1130 days (~37 months)
```

4b Count of transactions where the total amount was negative.

PYTHON

```
print('Answer:', (Customer_Final['total_amt'] < 0).sum(), 'transactions (these are
returns)')
```

RESULT ON THE REAL CASE DATA

```
Answer: 2177 transactions (these are returns)
```

5 Which product categories are more popular among female vs male customers?

PYTHON

```
out = Customer_Final[Customer_Final['total_amt'] > 0].pivot_table(index='prod_cat',
columns='Gender', values='Qty', aggfunc='sum')
out['Leader'] = np.where(out['M'] > out['F'], 'Male', 'Female')
out.plot.bar(y=['F', 'M'], title='Units bought by category and gender', rot=0)
```

RESULT ON THE REAL CASE DATA

Gender	F	M	Leader
prod_cat			
Bags	2693	2657	Female

Gender	F	M	Leader
prod_cat			
Books	7952	8446	Male
Clothing	3888	4175	Male
Electronics	6405	7153	Male
Footwear	4158	3968	Female
Home and kitchen	5482	5699	Male

6 Which city code has the maximum customers, and what share of customers is from that city?

PYTHON

```
cc = Customer_Final.groupby('city_code')['cust_id'].nunique().sort_values(ascending=False)
top = cc.index[0]
print(f'Answer: city code {int(top)} with {cc.iloc[0]} customers =
{cc.iloc[0]/cc.sum()*100:.2f}% of customers')
out = cc.rename('unique_customers')
```

RESULT ON THE REAL CASE DATA

```
Answer: city code 3 with 576 customers = 10.47% of customers
```

city_code	unique_customers
3.0	576
5.0	570
4.0	569
7.0	563
8.0	551
2.0	546
10.0	546
1.0	535
9.0	532

city_code	unique_customers
6.0	516

7 Which store type sells the maximum products by value and by quantity?

PYTHON

```
out = Customer_Final.groupby('Store_type')[['Qty',
'total_amt']].sum().sort_values('total_amt', ascending=False)
print('Answer (by value):', out['total_amt'].idxmax(), '| (by quantity):',
out['Qty'].idxmax())
```

RESULT ON THE REAL CASE DATA

```
Answer (by value): e-Shop | (by quantity): e-Shop
```

Store_type	Qty	total_amt
e-Shop	22763	19,824,816.05
Flagship store	11133	9,715,688.19
MBR	11194	9,674,486.05
TeleShop	10984	9,364,781.07

8 Total amount earned from Electronics and Clothing categories at Flagship stores.

PYTHON

```
m = (Customer_Final['Store_type'] == 'Flagship store') &
Customer_Final['prod_cat'].isin(['Electronics', 'Clothing'])
print(f"Answer: {Customer_Final.loc[m, 'total_amt'].sum():,.2f} (net of returns)")
```

RESULT ON THE REAL CASE DATA

```
Answer: 3,409,559.27 (net of returns)
```

9 Total amount earned from Male customers under the Electronics category.

PYTHON

```python
m = (Customer_Final['Gender'] == 'M') & (Customer_Final['prod_cat'] == 'Electronics')
print(f"Answer: {Customer_Final.loc[m, 'total_amt'].sum():,.2f}")
```

RESULT ON THE REAL CASE DATA

```
Answer: 5,703,109.42
```

10 How many customers have more than 10 unique transactions, after removing transactions with negative amounts?

PYTHON

```python
pos = Customer_Final[Customer_Final['total_amt'] > 0]
n = pos.groupby('cust_id')['transaction_id'].nunique()
print('Answer:', (n > 10).sum(), 'customers')
```

RESULT ON THE REAL CASE DATA

```
Answer: 6 customers
```

11 For customers aged 25–35: (a) total spend on Electronics and Books; (b) total spend between 1 Jan 2014 and 1 Mar 2014.

PYTHON

```python
ref = Customer_Final['tran_date'].max()   # age measured on the latest transaction date
Customer_Final['age'] = (ref - Customer_Final['DOB']).dt.days / 365.25
grp = Customer_Final[Customer_Final['age'].between(25, 35)]
a = grp[grp['prod_cat'].isin(['Electronics', 'Books'])]['total_amt'].sum()
b = grp[grp['tran_date'].between('2014-01-01', '2014-03-01')]['total_amt'].sum()
print(f'Answer (a): {a:,.2f}')
print(f'Answer (b): {b:,.2f}')
```

RESULT ON THE REAL CASE DATA

```
Answer (a): 10,206,666.21
Answer (b): 1,000,693.53
```

Insight Age is computed as of the latest transaction date in the data. Using today's date instead would shift everyone's age and change the answer, so state your reference date in the submission.

Python Case Study 2: Exploratory Analysis of Credit Card Data

PSPD Bank operates in 50+ countries. The CEO wants to understand customer spend and repayment behaviour to evaluate bankruptcy, fraud and collections risk and to make proactive offers.

Data

Customer Acquisition (100 customers: age, city, product, per-transaction limit, segment) · Spend (1,500 transactions by type) · Repayment (1,523 payments).

Tool and module

Python (pandas, matplotlib)
Python for Data Science

1a Replace age below 18 with the mean age.

PYTHON

```
print('Customers under 18 before:', (acq['Age'] < 18).sum())
acq.loc[acq['Age'] < 18, 'Age'] = acq['Age'].mean()
print('Customers under 18 after :', (acq['Age'] < 18).sum())
```

RESULT ON THE REAL CASE DATA

```
Customers under 18 before: 6
Customers under 18 after : 0
```

1b If spend is more than the customer's limit, replace it with 50% of that customer's limit.

PYTHON

```
spend = spend.merge(acq[['Customer', 'Limit']], on='Customer', how='left')
over = spend['Amount'] > spend['Limit']
print('Spend rows above limit:', over.sum())
spend.loc[over, 'Amount'] = spend.loc[over, 'Limit'] * 0.5
```

RESULT ON THE REAL CASE DATA

```
Spend rows above limit: 607
```

1c If repayment is more than the limit, replace it with the limit.

PYTHON

```
rep = rep.merge(acq[['Customer', 'Limit']], on='Customer', how='left')
over = rep['Amount'] > rep['Limit']
print('Repayment rows above limit:', over.sum())
rep.loc[over, 'Amount'] = rep.loc[over, 'Limit']
```

RESULT ON THE REAL CASE DATA

```
Repayment rows above limit: 600
```

2a How many distinct customers exist?

PYTHON

```
print('Answer:', acq['Customer'].nunique())
```

RESULT ON THE REAL CASE DATA

```
Answer: 100
```

2b How many distinct spend categories exist?

PYTHON

```
print('Answer:', spend['Type'].nunique(), '->', sorted(spend['Type'].unique()))
```

RESULT ON THE REAL CASE DATA

```
Answer: 15 -> ['AIR TICKET', 'AUTO', 'BIKE', 'BUS TICKET', 'CAMERA', 'CAR', 'CLOTHES',
'FOOD', 'JEWELLERY', 'MOVIE TICKET', 'PETRO', 'RENTAL', 'SANDALS', 'SHOPPING', 'TRAIN
TICKET']
```

2c Average monthly spend by customers.

PYTHON

```
spend['YM'] = spend['Month'].dt.to_period('M')
ms = spend.groupby(['Customer', 'YM'])['Amount'].sum()
print(f'Answer: average monthly spend per customer = {ms.mean():,.2f}')
```

RESULT ON THE REAL CASE DATA

```
Answer: average monthly spend per customer = 272,343.35
```

2d Average monthly repayment by customers.

PYTHON

```python
rep['YM'] = rep['Month'].dt.to_period('M')
mr = rep.groupby(['Customer', 'YM'])['Amount'].sum()
print(f'Answer: average monthly repayment per customer = {mr.mean():,.2f}')
```

RESULT ON THE REAL CASE DATA

```
Answer: average monthly repayment per customer = 306,995.44
```

2e If the monthly interest rate is 2.9%, what is the bank's profit for each month? (Profit = interest on positive (repayment – spend).)

PYTHON

```python
m = pd.concat([rep.groupby('YM')['Amount'].sum().rename('Repayment'),
               spend.groupby('YM')['Amount'].sum().rename('Spend')], axis=1).fillna(0)
m['Monthly_Profit'] = m['Repayment'] - m['Spend']
m['Interest_Earned'] = np.where(m['Monthly_Profit'] > 0, m['Monthly_Profit'] * 0.029, 0)
out = m
print(f"Total interest earned across months: {m['Interest_Earned'].sum():,.2f}")
```

RESULT ON THE REAL CASE DATA

```
Total interest earned across months: 998,733.65
```

YM	Repayment	Spend	Monthly_Profit	Interest_Earned
2004-01	44,792,020.72	44,279,811.96	512,208.76	14,854.05
2004-02	3,391,537.65	3,835,951.84	-444,414.19	0.00
2004-03	2,666,519.22	1,871,913.27	794,605.95	23,043.57
2004-04	5,268,979.74	3,489,415.34	1,779,564.40	51,607.37
2004-05	3,782,759.91	2,146,577.36	1,636,182.55	47,449.29
2004-09	1,902,816.41	2,402,109.69	-499,293.28	0.00
2004-11	1,788,398.63	1,114,989.10	673,409.53	19,528.88

YM	Repayment	Spend	Monthly_Profit	Interest_Earned
2005-01	4,894,040.25	5,383,606.78	-489,566.53	0.00
2005-02	25,491,800.20	25,928,183.21	-436,383.01	0.00
2005-04	3,634,903.56	4,100,065.73	-465,162.17	0.00
2005-05	30,818,764.69	21,647,612.42	9,171,152.27	265,963.42
2005-06	3,493,885.53	8,010,511.98	-4,516,626.45	0.00
2005-07	1,799,339.14	1,691,243.28	108,095.86	3,134.78
2005-08	5,549,411.88	3,967,808.19	1,581,603.69	45,866.51
2005-09	882,719.54	1,531,932.87	-649,213.33	0.00

Showing first 15 of 30 rows.

Insight Calculated at the bank level for each calendar month. Some classes calculate it per customer per month first and then sum only positive values; mention which approach you used.

2f Top 5 product types (spend categories).

PYTHON

```
out = spend.groupby('Type')['Amount'].sum().sort_values(ascending=False).head(5)
```

RESULT ON THE REAL CASE DATA

Type	Amount
PETRO	28,597,384.98
CAMERA	27,690,738.44
FOOD	20,519,243.60
AIR TICKET	20,155,847.12
TRAIN TICKET	19,995,825.72

2g Which city has the maximum spend?

PYTHON

```
s = spend.merge(acq[['Customer', 'City']], on='Customer').groupby('City')
['Amount'].sum().sort_values(ascending=False)
print('Answer:', s.index[0]); out = s
```

RESULT ON THE REAL CASE DATA

```
Answer: COCHIN
```

City	Amount
COCHIN	45,963,513.50
CALCUTTA	41,108,132.30
BANGALORE	40,461,647.02
BOMBAY	28,603,127.38
CHENNAI	22,128,635.65
TRIVANDRUM	15,337,155.87
PATNA	15,260,586.97
DELHI	10,918,286.54

2h Which age group spends the most?

PYTHON

```
acq['AgeGroup'] = pd.cut(acq['Age'], bins=[18, 30, 40, 50, 60, 70, 90], right=False)
s = spend.merge(acq[['Customer', 'AgeGroup']], on='Customer').groupby('AgeGroup',
observed=True)['Amount'].sum()
print('Answer:', s.idxmax()); out = s
```

RESULT ON THE REAL CASE DATA

```
Answer: [18, 30)
```

AgeGroup	Amount
[18, 30)	52,689,898.21
[30, 40)	37,329,313.58
[40, 50)	48,965,558.48

AgeGroup	Amount
[50, 60)	31,438,978.56
[60, 70)	20,592,971.57
[70, 90)	28,764,364.83

2i Top 10 customers by repayment.

PYTHON

```
out = rep.groupby('Customer')['Amount'].sum().sort_values(ascending=False).head(10)
```

RESULT ON THE REAL CASE DATA

Customer	Amount
A61	10,539,142.91
A60	9,876,290.74
A13	9,572,000.66
A43	8,489,871.46
A45	8,448,334.87
A12	8,334,760.16
A14	7,943,268.63
A44	7,744,730.12
A39	7,622,483.30
A42	7,615,460.86

3 City-wise spend on each product per year, with a chart.

PYTHON

```
s = spend.merge(acq[['Customer', 'City', 'Product']], on='Customer')
s['Year'] = s['Month'].dt.year
out = s.pivot_table(index=['City', 'Year'], columns='Product', values='Amount',
aggfunc='sum').round(0)
```

```
s.pivot_table(index='City', columns='Product', values='Amount',
aggfunc='sum').plot.bar(title='Spend by city and card product', rot=30)
```

RESULT ON THE REAL CASE DATA

City	Year	Product Gold	Platimum	Silver
BANGALORE	2004	9,289,879.00	1,112,732.00	1,291,853.00
	2005	12,892,363.00	912,285.00	830,612.00
	2006	12,030,611.00	685,831.00	1,415,481.00
BOMBAY	2004	6,987,854.00	897,265.00	532,089.00
	2005	8,983,018.00	711,805.00	508,989.00
	2006	7,917,144.00	972,130.00	1,092,834.00
CALCUTTA	2004	7,477,141.00	2,037,690.00	500,006.00
	2005	10,303,356.00	4,434,696.00	440,147.00
	2006	11,167,533.00	3,589,607.00	1,157,956.00
CHENNAI	2004	1,059,618.00	3,535,652.00	282,057.00
	2005	3,740,946.00	5,053,956.00	377,184.00
	2006	2,704,289.00	5,024,923.00	350,010.00
COCHIN	2004	7,315,850.00	3,419,904.00	814,230.00
	2005	12,110,613.00	4,960,922.00	652,269.00
	2006	10,499,142.00	4,920,592.00	1,269,991.00

Showing first 15 of 24 rows.

4a Monthly comparison of total spend, city-wise.

PYTHON

```
s = spend.merge(acq[['Customer', 'City']], on='Customer')
s['Mon'] = s['Month'].dt.month
s.pivot_table(index='Mon', columns='City', values='Amount',
aggfunc='sum').plot(title='Monthly spend by city', marker='o', figsize=(9, 4))
```

RESULT ON THE REAL CASE DATA

4b Yearly spend on air tickets.

PYTHON

```
a = spend[spend['Type'] == 'AIR TICKET'].groupby(spend['Month'].dt.year)['Amount'].sum()
a.plot.bar(title='Yearly spend on air tickets', color='#E9A23B', rot=0); out = a
```

RESULT ON THE REAL CASE DATA

Month	Amount
2004	4,357,225.35
2005	10,550,152.21
2006	5,248,469.56

4c Monthly spend for each product: is there seasonality?

PYTHON

```
s = spend.merge(acq[['Customer', 'Product']], on='Customer')
s.pivot_table(index=s['Month'].dt.month, columns='Product', values='Amount',
aggfunc='sum').plot(marker='o', title='Spend by calendar month and product')
```

RESULT ON THE REAL CASE DATA

5 UDF: top 10 customers per city by repayment, for a chosen product (Gold/Silver/Platinum) and time period (yearly or monthly).

PYTHON

```
def top10_repayers(product, period='yearly'):
    r = rep.merge(acq[['Customer', 'City', 'Product']], on='Customer')
    r = r[r['Product'].str.lower().str.startswith(product.lower()[:4])]   # data spells it
'Platimum'
    r['Period'] = r['Month'].dt.year if period == 'yearly' else
r['Month'].dt.to_period('M')
    g = r.groupby(['City', 'Period', 'Customer'])['Amount'].sum().reset_index()
```

```
    g['Rank'] = g.groupby(['City', 'Period'])['Amount'].rank(ascending=False,
method='first')
    return g[g['Rank'] <= 10].sort_values(['City', 'Period', 'Rank'])

out = top10_repayers('Gold', 'yearly')
```

RESULT ON THE REAL CASE DATA

	City	Period	Customer	Amount	Rank
2	BANGALORE	2004	A14	2,770,261.30	1.00
0	BANGALORE	2004	A1	2,359,797.84	2.00
1	BANGALORE	2004	A13	2,337,012.77	3.00
4	BANGALORE	2004	A43	1,906,497.47	4.00
3	BANGALORE	2004	A30	978,777.17	5.00
5	BANGALORE	2004	A63	788,855.60	6.00
10	BANGALORE	2005	A43	4,006,956.60	1.00
7	BANGALORE	2005	A13	3,757,673.00	2.00
9	BANGALORE	2005	A30	2,667,746.38	3.00
8	BANGALORE	2005	A14	2,491,981.61	4.00
6	BANGALORE	2005	A1	1,529,158.77	5.00
12	BANGALORE	2006	A13	3,477,314.89	1.00
13	BANGALORE	2006	A14	2,681,025.72	2.00
15	BANGALORE	2006	A43	2,576,417.39	3.00
14	BANGALORE	2006	A30	2,486,958.41	4.00

Showing first 15 of 88 rows.

Insight The product column spells Platinum as 'Platimum', so the function matches on the first four letters. Mention this data-quality issue in your write-up.

Python Case Study 3: EDA and Hypothesis Testing on Insurance Claims

An insurer wants a 360-degree view of claims and customers, to clean the data, profile claims by segment, and test business hypotheses about claim amounts with statistical tests.

Data

claims.csv (1,100 claims: cause, date, area, police report, type, amount with $ sign, total policy claims, fraud flag) · cust_demographics.csv (1,085 customers: gender, DOB, state, segment).

Tool and module

Python (pandas, scipy)
Python for Data Science

1 Combine claims and customer data into a 360-degree view.

PYTHON

```
df = claims.merge(cust, how='inner', left_on='customer_id', right_on='CUST_ID')
print('claims:', claims.shape, '| customers:', cust.shape, '| merged:', df.shape)
```

RESULT ON THE REAL CASE DATA

```
claims: (1100, 10) | customers: (1085, 6) | merged: (1085, 16)
```

2 Data audit: are any data types mismatched with their business meaning?

PYTHON

```
out = df.dtypes.astype(str).rename('dtype')
print('Mismatches: claim_amount is text ($ sign), claim_date and DateOfBirth are text not
dates, IDs are numbers but should be treated as labels.')
```

RESULT ON THE REAL CASE DATA

```
Mismatches: claim_amount is text ($ sign), claim_date and DateOfBirth are text not dates,
IDs are numbers but should be treated as labels.
```

	dtype
claim_id	int64

	dtype
customer_id	int64
incident_cause	str
claim_date	str
claim_area	str
police_report	str
claim_type	str
claim_amount	str
total_policy_claims	float64
fraudulent	str
CUST_ID	int64
gender	str
DateOfBirth	str
State	str
Contact	str

Showing first 15 of 16 rows.

3 Convert claim_amount to numeric by removing the $ sign.

PYTHON

```
df['claim_amount'] = pd.to_numeric(df['claim_amount'].str.replace('$', '', regex=False),
errors='coerce')
df['claim_date'] = pd.to_datetime(df['claim_date'], format='%m/%d/%Y')
dob = pd.to_datetime(df['DateOfBirth'], format='%d-%b-%y')
df['DateOfBirth'] = dob.where(dob.dt.year <= 2018, dob - pd.DateOffset(years=100))   # '79
-> 1979, not 2079
print(df[['claim_amount', 'claim_date', 'DateOfBirth']].dtypes.to_string())
```

RESULT ON THE REAL CASE DATA

```
claim_amount           float64
claim_date      datetime64[us]
DateOfBirth     datetime64[us]
```

4 Create an alert flag (1/0) for injury claims not reported to police.

PYTHON

```
df['unreported_injury_flag'] = np.where((df['claim_type'].str.contains('Injury')) &
(df['police_report'] == 'No'), 1, 0)
print('Answer:', df['unreported_injury_flag'].sum(), 'injury claims were not reported to
police')
```

RESULT ON THE REAL CASE DATA

```
Answer: 60 injury claims were not reported to police
```

5 Keep the most recent claim per customer and drop duplicates on customer ID.

PYTHON

```
before = len(df)
df = df.sort_values('claim_date').drop_duplicates('customer_id', keep='last')
print(f'Rows before: {before} | after: {len(df)} | duplicates removed: {before-len(df)}')
```

RESULT ON THE REAL CASE DATA

```
Rows before: 1085 | after: 1078 | duplicates removed: 7
```

6 Impute missing values (mean for continuous, mode for categorical).

PYTHON

```
print('Missing before:'); print(df.isna().sum()[df.isna().sum() > 0].to_string())
for col in df.columns:
    if df[col].isna().any():
        df[col] = df[col].fillna(df[col].mean() if df[col].dtype.kind in 'fi' else
df[col].mode()[0])
print('Missing after:', int(df.isna().sum().sum()))
```

RESULT ON THE REAL CASE DATA

```
Missing before:
claim_amount          65
total_policy_claims    10
Missing after: 0
```

7 Calculate age in years and categorise: Children <18, Youth 18–30, Adult 30–60, Senior >60.

PYTHON

```python
ref = pd.Timestamp('2018-10-01')
df['age'] = ((ref - df['DateOfBirth']).dt.days / 365.25).astype(int)
df['age_group'] = pd.cut(df['age'], [0, 18, 30, 60, 200], labels=['Children', 'Youth',
'Adult', 'Senior'], right=False)
out = df['age_group'].value_counts().sort_index()
```

RESULT ON THE REAL CASE DATA

age_group	count
Children	0
Youth	310
Adult	768
Senior	0

Insight Two-digit birth years such as '79' are read by Python as 2079. They are moved back 100 years before computing age, otherwise ages come out negative.

8 Average claim amount by customer segment.

PYTHON

```python
out = df.groupby('Segment')['claim_amount'].mean().round(2)
```

RESULT ON THE REAL CASE DATA

Segment	claim_amount
Gold	12,846.40
Platinum	12,371.65
Silver	12,267.86

9 Total claim amount by incident cause, for claims at least 20 days before 1 Oct 2018.

PYTHON

```python
out = df[df['claim_date'] <= pd.Timestamp('2018-10-01') -
pd.Timedelta(days=20)].groupby('incident_cause')['claim_amount'].sum()
```

RESULT ON THE REAL CASE DATA

incident_cause	claim_amount
Crime	721,938.63
Driver error	3,315,850.72
Natural causes	1,312,987.04
Other causes	3,723,151.80
Other driver error	3,277,536.45

10 How many adults from TX, DE and AK claimed for driver-related causes?

PYTHON

```
m = (df['age_group'] == 'Adult') & df['State'].isin(['TX', 'DE', 'AK']) &
df['incident_cause'].str.contains('driver', case=False)
print('Answer:', m.sum()); out = df[m].groupby('State').size().rename('claims')
```

RESULT ON THE REAL CASE DATA

```
Answer: 25
```

State	claims
AK	8
DE	10
TX	7

11 Pie chart of total claim amount by gender and segment (show %).

PYTHON

```
p = df.groupby(['gender', 'Segment'])['claim_amount'].sum()
p.plot.pie(autopct='%1.1f%%', figsize=(6, 6), ylabel='', title='Claim amount by gender and
segment'); out = p
```

RESULT ON THE REAL CASE DATA

gender	Segment	claim_amount
Female	Gold	2,110,044.54
	Platinum	2,369,722.09
	Silver	1,897,544.95
Male	Gold	2,655,969.95
	Platinum	2,096,445.31
	Silver	2,347,134.40

12 Which gender claimed the most for driver-related issues?

PYTHON

```
g = df[df['incident_cause'].str.contains('driver', case=False)].groupby('gender')
['claim_amount'].agg(['count', 'sum'])
g['count'].plot.bar(rot=0, title='Driver-related claims by gender'); out = g
print('Answer:', g['count'].idxmax(), 'by number of claims')
```

RESULT ON THE REAL CASE DATA

```
Answer: Male by number of claims
```

gender	count	sum
Female	221	3,136,717.95
Male	281	3,975,411.85

13 Which age group had the most fraudulent claims? Show a bar chart.

PYTHON

```
f = df[df['fraudulent'] == 'Yes']['age_group'].value_counts().sort_index()
f.plot.bar(rot=0, color='#C8553D', title='Fraudulent claims by age group'); out = f
print('Answer:', f.idxmax())
```

RESULT ON THE REAL CASE DATA

```
Answer: Adult
```

age_group	count
Children	0
Youth	68
Adult	177
Senior	0

14 Monthly trend of total claim amount (months in chronological order).

PYTHON

```
t = df.groupby(df['claim_date'].dt.to_period('M'))['claim_amount'].sum()
t.index = t.index.to_timestamp(); t.plot(marker='o', title='Total claim amount by month');
out = t
```

RESULT ON THE REAL CASE DATA

claim_date	claim_amount
2017-01-01	608,016.18
2017-02-01	631,456.45
2017-03-01	615,576.45
2017-04-01	747,975.73
2017-05-01	546,741.68
2017-06-01	560,143.73
2017-07-01	790,868.91
2017-08-01	431,616.45
2017-09-01	459,469.09
2017-10-01	608,989.18
2017-11-01	601,187.45

claim_date	claim_amount
2017-12-01	694,252.73
2018-01-01	719,154.41
2018-02-01	533,085.95
2018-03-01	704,673.68

Showing first 15 of 22 rows.

15 Average claim by gender and age group, faceted by fraudulent vs non-fraudulent.

PYTHON

```
fig, ax = plt.subplots(1, 2, figsize=(11, 4), sharey=True)
for a, flag in zip(ax, ['No', 'Yes']):
    df[df['fraudulent'] == flag].pivot_table(index='age_group', columns='gender',
values='claim_amount', aggfunc='mean', observed=True).plot.bar(ax=a, rot=0,
title=f'Fraudulent = {flag}')
```

RESULT ON THE REAL CASE DATA

16 Hypothesis: is the amount claimed by males and females similar?

PYTHON

```
m, f = df.loc[df.gender == 'Male', 'claim_amount'], df.loc[df.gender == 'Female',
'claim_amount']
t, p = stats.ttest_ind(m, f, equal_var=False)
print('H0: mean claim (male) = mean claim (female)  |  test: two-sample t-test (Welch)')
print(f't = {t:.3f}, p = {p:.4f} ->', 'Reject H0' if p < 0.05 else 'Fail to reject H0: no
significant difference')
```

RESULT ON THE REAL CASE DATA

```
H0: mean claim (male) = mean claim (female)  |  test: two-sample t-test (Welch)
t = 0.963, p = 0.3357 -> Fail to reject H0: no significant difference
```

17 Hypothesis: is there a relationship between age category and segment?

PYTHON

```
ct = pd.crosstab(df['age_group'], df['Segment'])
chi2, p, dof, _ = stats.chi2_contingency(ct)
print('H0: age group and segment are independent  |  test: chi-square test of
independence')
print(f'chi2 = {chi2:.3f}, dof = {dof}, p = {p:.4f} ->', 'Reject H0' if p < 0.05 else
'Fail to reject H0: no relationship'); out = ct
```

RESULT ON THE REAL CASE DATA

```
H0: age group and segment are independent  |  test: chi-square test of independence
chi2 = 1.818, dof = 2, p = 0.4029 -> Fail to reject H0: no relationship
```

Segment	Gold	Platinum	Silver
age_group			
Youth	100	113	97
Adult	271	248	249

18 Hypothesis: has the current year shown a significant rise in claim amounts vs the 2016-17 average of $10,000?

PYTHON

```
cur = df.loc[df['claim_date'].dt.year == 2018, 'claim_amount']
t, p2 = stats.ttest_1samp(cur, 10000)
p = p2 / 2 if t > 0 else 1 - p2 / 2      # one-tailed (greater than)
print('H0: mean 2018 claim <= 10,000   H1: mean 2018 claim > 10,000  |  test: one-sample
t-test, right-tailed')
print(f'mean 2018 = {cur.mean():,.2f}, t = {t:.3f}, one-tailed p = {p:.4f} ->', 'Reject
H0: significant rise' if p < 0.05 else 'Fail to reject H0: no significant rise')
```

RESULT ON THE REAL CASE DATA

```
H0: mean 2018 claim <= 10,000   H1: mean 2018 claim > 10,000  |  test: one-sample t-test,
right-tailed
mean 2018 = 12,743.44, t = 4.477, one-tailed p = 0.0000 -> Reject H0: significant rise
```

19 Hypothesis: is there a difference in claim amounts between age groups?

PYTHON

```python
groups = [g['claim_amount'].values for _, g in df.groupby('age_group', observed=True) if len(g) > 1]
F, p = stats.f_oneway(*groups)
print('H0: mean claim is equal across age groups  |  test: one-way ANOVA')
print(f'F = {F:.3f}, p = {p:.4f} ->', 'Reject H0' if p < 0.05 else 'Fail to reject H0: no significant difference')
```

RESULT ON THE REAL CASE DATA

```
H0: mean claim is equal across age groups  |  test: one-way ANOVA
F = 0.746, p = 0.3881 -> Fail to reject H0: no significant difference
```

20 Hypothesis: is there a relationship between total number of policy claims and claim amount?

PYTHON

```python
r, p = stats.pearsonr(df['total_policy_claims'], df['claim_amount'])
print('H0: no linear correlation (rho = 0)  |  test: Pearson correlation')
print(f'r = {r:.3f}, p = {p:.4f} ->', 'Reject H0' if p < 0.05 else 'Fail to reject H0: no meaningful relationship')
```

RESULT ON THE REAL CASE DATA

```
H0: no linear correlation (rho = 0)  |  test: Pearson correlation
r = -0.022, p = 0.4724 -> Fail to reject H0: no meaningful relationship
```

Python Case Study 4: Visualisations on Pharma Sales Data

A leading pharmaceutical company wants a dynamic view of sales KPIs at national, region and territory level for 2015 vs 2016, to see progress and failures and speed up decisions. Reshape the data with groupby, stack/unstack and pivot as needed.

Data

SalesData.csv (3,709 rows: account, region, division, state, tier, month, Sales and Units for 2015 and 2016).

Tool and module

Python (pandas, matplotlib)
Python for Data Science

1 Compare sales by region for 2016 vs 2015 with a bar chart.

PYTHON

```
out = sales.groupby('Region')[['Sales2015', 'Sales2016']].sum()
out.plot.bar(rot=0, title='Sales by region: 2015 vs 2016')
```

RESULT ON THE REAL CASE DATA

Region	Sales2015	Sales2016
Central	7,891,728.86	9,787,808.73
East	9,512,916.49	12,667,230.45
West	5,349,744.64	7,209,689.43

2 Contribution of each region to 2016 sales (pie chart).

PYTHON

```
s = sales.groupby('Region')['Sales2016'].sum()
s.plot.pie(autopct='%1.1f%%', ylabel='', figsize=(5.5, 5.5), title='Share of 2016 sales by region'); out = s
```

RESULT ON THE REAL CASE DATA

Region	Sales2016
Central	9,787,808.73
East	12,667,230.45
West	7,209,689.43

3 Compare total sales of 2015 and 2016 by Region and Tier.

PYTHON

```python
out = sales.groupby(['Region', 'Tier'])[['Sales2015', 'Sales2016']].sum()
out.plot.bar(figsize=(11, 4), title='Sales by region and tier')
```

RESULT ON THE REAL CASE DATA

Region	Tier	Sales2015	Sales2016
Central	High	4,798,698.43	6,026,042.94
	Low	943,439.67	1,132,832.85
	Med	2,068,225.87	2,632,181.47
	Out	81,364.89	-3,248.53
East	High	6,102,946.12	7,817,151.10
	Low	901,665.80	1,144,929.59
	Med	2,470,998.02	3,705,149.76
	Out	37,306.55	0.00
West	High	2,944,789.26	3,768,038.35
	Low	671,064.38	1,099,502.18
	Med	1,718,475.92	2,342,148.90
	Out	15,415.08	0.00

4 In the East region, which state declined in 2016 vs 2015?

PYTHON

```
e = sales[sales['Region'] == 'East'].groupby('State')[['Sales2015', 'Sales2016']].sum()
e['Change'] = e['Sales2016'] - e['Sales2015']; out = e
print('Answer:', list(e[e['Change'] < 0].index))
e[['Sales2015', 'Sales2016']].plot.bar(rot=0, title='East region sales by state')
```

RESULT ON THE REAL CASE DATA

```
Answer: ['NY']
```

State	Sales2015	Sales2016	Change
CT	197,202.79	323,502.31	126,299.52
DC	216,723.95	257,421.69	40,697.74
FL	1,660,162.00	2,508,232.28	848,070.28
GA	681,546.46	946,919.77	265,373.31
MA	419,278.75	687,096.45	267,817.70
MD	527,309.06	750,008.79	222,699.73
ME	77,425.64	183,673.05	106,247.41
NC	1,292,801.64	1,610,521.00	317,719.36
NH	136,419.38	144,717.51	8,298.13
NJ	520,419.29	782,289.19	261,869.90
NY	1,725,415.06	1,703,135.80	-22,279.26
PA	785,092.76	894,319.49	109,226.73
RI	156,311.80	162,454.65	6,142.85
SC	247,811.78	463,414.12	215,602.34
TN	49,677.00	70,109.41	20,432.41

Showing first 15 of 16 rows.

5 In the High tier, which divisions sold fewer units in 2016 than 2015?

PYTHON

```python
h = sales[sales['Tier'] == 'High'].groupby('Division')[['Units2015', 'Units2016']].sum()
h['Change'] = h['Units2016'] - h['Units2015']; out = h
dec = list(h[h['Change'] < 0].index)
print('Answer:', dec if dec else 'None: every division in the High tier sold more units in 2016 than in 2015')
h[['Units2015', 'Units2016']].plot.bar(rot=45, title='High tier units by division')
```

RESULT ON THE REAL CASE DATA

```
Answer: None: every division in the High tier sold more units in 2016 than in 2015
```

Division	Units2015	Units2016	Change
BIG APPLE	180.83	231.00	50.17
CHARGERS	123.67	164.50	40.83
CONGRESSIONAL	164.50	205.00	40.50
DIAMONDBACK	153.33	176.67	23.33
EMPIRE	414.33	449.33	35.00
GRIDIRON	213.67	246.17	32.50
GULF STREAM	185.00	301.00	116.00
HAILSTORM	187.83	247.00	59.17
KINETIC	214.33	244.50	30.17
MINUTEMEN	215.00	267.50	52.50
MUDDY WATERS	150.67	211.50	60.83
NORDIC	210.17	225.83	15.67
NORTHWEST	135.50	163.67	28.17
ORION	175.83	276.00	100.17
RACEWAY	178.33	297.00	118.67

Showing first 15 of 26 rows.

6 Create a Qtr column from Month (Jan–Mar Q1, Apr–Jun Q2, Jul–Sep Q3, Oct–Dec Q4).

PYTHON

```
q = {'Jan': 'Q1', 'Feb': 'Q1', 'Mar': 'Q1', 'Apr': 'Q2', 'May': 'Q2', 'Jun': 'Q2',
     'Jul': 'Q3', 'Aug': 'Q3', 'Sep': 'Q3', 'Oct': 'Q4', 'Nov': 'Q4', 'Dec': 'Q4'}
sales['Qtr'] = sales['Month'].map(q)
out = sales[['Month', 'Qtr']].drop_duplicates().sort_values('Qtr')
```

RESULT ON THE REAL CASE DATA

	Month	Qtr
3	Mar	Q1
6	Feb	Q1
31	Jan	Q1
2	Jun	Q2
18	Apr	Q2
22	May	Q2
0	Aug	Q3
4	Sep	Q3
5	Jul	Q3
1	Oct	Q4
20	Nov	Q4
25	Dec	Q4

7 Compare quarter-wise sales for 2015 and 2016 (bar plot).

PYTHON

```
out = sales.groupby('Qtr')[['Sales2015', 'Sales2016']].sum()
out.plot.bar(rot=0, title='Quarterly sales 2015 vs 2016')
```

RESULT ON THE REAL CASE DATA

Qtr	Sales2015	Sales2016
Q1	5,485,800.16	6,997,953.24
Q2	5,390,862.08	7,237,361.17
Q3	6,164,093.84	7,861,546.02
Q4	5,713,633.91	7,567,868.17

8 Composition of 2016 quarterly sales by tier: four pie charts, one per quarter.

PYTHON

```
fig, ax = plt.subplots(1, 4, figsize=(14, 3.8))
for a, qt in zip(ax, ['Q1', 'Q2', 'Q3', 'Q4']):
    v = sales[sales['Qtr'] == qt].groupby('Tier')['Sales2016'].sum()
    v[v > 0].plot.pie(ax=a, autopct='%1.0f%%', ylabel='', title=qt)   # 'Out' tier is 0 or
negative (returns), so it is left out
```

RESULT ON THE REAL CASE DATA

Insight The 'Out' tier has zero or slightly negative 2016 sales (returns), and a pie chart cannot show negative slices, so only Low, Med and High appear.

Basic Statistics Case Study: Hypothesis Testing (5 business problems)

Five short business problems, each answered with the right statistical test: Lending Club interest rates, pricing-expert quotes, a treatment-facility re-engineering programme, software job prioritisation, and a Film on the Rocks customer survey.

Data

LoansData.csv (2,500 loans) · Price_Quotes.csv (12 orders, two experts) · Treatment_Facility.csv (20 months) · Priority_Assessment.csv (642 jobs) · Films.csv (330 survey responses).

Tool and module

Python (scipy)
Python for Data Science

1a Lending Club: does interest rate vary with loan amount (less interest for higher loans)?

PYTHON

```
d = loans.dropna(subset=['Amount.Requested'])
r, p = stats.pearsonr(d['Amount.Requested'], d['rate'])
print('H0: no correlation between amount requested and interest rate | Pearson
correlation')
print(f'r = {r:.3f},', verdict(p))
print('Reading: r is positive, so bigger loans carry slightly HIGHER rates, which
contradicts the claim of less interest for high amounts.')
```

RESULT ON THE REAL CASE DATA

```
H0: no correlation between amount requested and interest rate | Pearson correlation
r = 0.332, p = 0.0000 -> Reject H0 at alpha = 0.05
Reading: r is positive, so bigger loans carry slightly HIGHER rates, which contradicts the
claim of less interest for high amounts.
```

1b Lending Club: does loan length affect interest rate?

PYTHON

```
a = loans.loc[loans['Loan.Length'] == '36 months', 'rate']; b =
loans.loc[loans['Loan.Length'] == '60 months', 'rate']
t, p = stats.ttest_ind(a, b, equal_var=False)
```

```
print('H0: mean rate (36 months) = mean rate (60 months) | two-sample t-test')
print(f'mean 36m = {a.mean():.2f}%, mean 60m = {b.mean():.2f}%, t = {t:.2f},', verdict(p))
```

RESULT ON THE REAL CASE DATA

```
H0: mean rate (36 months) = mean rate (60 months) | two-sample t-test
mean 36m = 12.13%, mean 60m = 16.41%, t = -21.95, p = 0.0000 -> Reject H0 at alpha = 0.05
```

1c Lending Club: does interest rate vary by loan purpose?

PYTHON

```
g = [x['rate'].values for _, x in loans.groupby('Loan.Purpose')]
F, p = stats.f_oneway(*g)
print('H0: mean rate is the same for every loan purpose | one-way ANOVA')
print(f'F = {F:.2f},', verdict(p))
out = loans.groupby('Loan.Purpose')['rate'].mean().sort_values().round(2)
```

RESULT ON THE REAL CASE DATA

```
H0: mean rate is the same for every loan purpose | one-way ANOVA
F = 7.48, p = 0.0000 -> Reject H0 at alpha = 0.05
```

Loan.Purpose	rate
renewable_energy	9.88
major_purchase	10.80
educational	11.01
car	11.11
home_improvement	11.59
medical	11.65
vacation	11.97
wedding	12.05
small_business	12.84
credit_card	13.05
other	13.16
house	13.45
debt_consolidation	13.59

Loan.Purpose	rate
moving	13.62

1d Lending Club: is there a relationship between FICO score and home ownership?

PYTHON

```
ct = pd.crosstab(loans['FICO.Range'], loans['Home.Ownership'])
chi2, p, dof, _ = stats.chi2_contingency(ct)
print('H0: FICO range and home ownership are independent | chi-square test of
independence')
print(f'chi2 = {chi2:.1f}, dof = {dof},', verdict(p))
loans['fico_low'] = loans['FICO.Range'].str[:3].astype(float)
out = loans.groupby('Home.Ownership')['fico_low'].agg(['count', 'mean']).round(1)
```

RESULT ON THE REAL CASE DATA

```
H0: FICO range and home ownership are independent | chi-square test of independence
chi2 = 472.8, dof = 148, p = 0.0000 -> Reject H0 at alpha = 0.05
```

Home.Ownership	count	mean
MORTGAGE	1145	711.70
NONE	1	800.00
OTHER	5	672.00
OWN	200	706.60
RENT	1144	700.10

Insight A chi-square test needs expected counts of at least 5 in most cells. OTHER and NONE have very few rows, so you can also re-run the test with only MORTGAGE, RENT and OWN and report both results.

2 Pricing experts: is there a difference between Mary's and Barry's average quotes?

PYTHON

```
pq = pd.read_csv('Price_Quotes.csv')
t, p = stats.ttest_rel(pq['Barry_Price'], pq['Mary_Price'])
print('H0: mean difference between paired quotes = 0 | paired t-test (same 12 orders
```

```
quoted by both)')
print(f"mean Barry = {pq['Barry_Price'].mean():.2f}, mean Mary =
{pq['Mary_Price'].mean():.2f}, t = {t:.3f},", verdict(p))
```

RESULT ON THE REAL CASE DATA

```
H0: mean difference between paired quotes = 0 | paired t-test (same 12 orders quoted by
both)
mean Barry = 124.33, mean Mary = 114.75, t = 2.521, p = 0.0284 -> Reject H0 at alpha =
0.05
```

Insight Use a **paired** t-test because both experts priced the same 12 orders. An independent two-sample test ignores that pairing and is the most common mistake in this question.

3 Treatment facility: did re-engineering change the critical-incident rate (and turnover)?

PYTHON

```
tf = pd.read_csv('Treatment_Facility.csv').rename(columns={'VAR4': 'TRFF_pct', 'VAR5':
'CI_pct'})
for col in ['CI_pct', 'Employee_Turnover']:
    pre, post = tf.loc[tf.Reengineer == 'Prior', col], tf.loc[tf.Reengineer == 'Post',
col]
    t, p = stats.ttest_ind(pre, post, equal_var=False)
    print(f'{col}: mean prior = {pre.mean():.2f}, mean post = {post.mean():.2f}, t =
{t:.2f},', verdict(p))
out = tf.groupby('Reengineer')[['Employee_Turnover', 'TRFF_pct',
'CI_pct']].mean().round(2)
```

RESULT ON THE REAL CASE DATA

```
CI_pct: mean prior = 53.89, mean post = 23.35, t = 2.21, p = 0.0456 -> Reject H0 at alpha
= 0.05
Employee_Turnover: mean prior = 11.74, mean post = 18.69, t = -1.57, p = 0.1521 -> Fail to
reject H0 at alpha = 0.05
```

Reengineer	Employee_Turnover	TRFF_pct	CI_pct
Post	18.69	9.23	23.35
Prior	11.74	20.54	53.89

Insight The file names its last two columns VAR4 and VAR5. From the brief they are TRFF (%) and CI (%), in that order. With only 13 months before and 7 after, the test has low power, so a non-significant p-value is weak evidence.

4 Job priorities: are high-priority jobs completed faster than medium, and medium faster than low?

PYTHON

```
pa = pd.read_csv('Priority_Assessment.csv')
print(pa.groupby('Priority')['Days'].agg(['count', 'mean',
'median']).round(2).to_string())
g = [x['Days'].values for _, x in pa.groupby('Priority')]
F, p = stats.f_oneway(*g)
print('\nH0: mean completion days are equal across priorities | one-way ANOVA')
print(f'F = {F:.3f},', verdict(p))
pa.boxplot(column='Days', by='Priority'); plt.suptitle(''); plt.title('Completion days by
priority')
```

RESULT ON THE REAL CASE DATA

```
          count  mean  median
Priority
High        326  3.02     0.6
Low          67  4.23     1.2
Medium      249  2.50     0.5

H0: mean completion days are equal across priorities | one-way ANOVA
F = 1.812, p = 0.1641 -> Fail to reject H0 at alpha = 0.05
```

Insight The averages run against the claim: medium-priority jobs (2.50 days) finish faster on average than high-priority jobs (3.02 days), and the ANOVA finds no significant difference. The prioritisation system is not working as intended. Completion times are right-skewed (means far above medians), so also confirm with a Kruskal–Wallis test.

5 Film on the Rocks survey: overall satisfaction, its drivers, patron profile and best media outlet.

PYTHON

```
f = pd.read_csv('Films.csv')
f['Gender'] = f['Gender'].replace({'1': 'Male', '2': 'Female'}); f['Marital_Status'] =
f['Marital_Status'].replace({'1': 'Married', '2': 'Single'})
lab = {1: 'Excellent', 2: 'Good', 3: 'Average', 4: 'Poor', 5: 'Very poor'}
print('Overall satisfaction (%):');
print((f['Overall'].map(lab).value_counts(normalize=True) * 100).round(1).to_string())
print('\nSpearman correlation of each factor with Overall (lower code = better):')
for col in ['Sinage', 'Parking', 'Clean']:
    d = f[[col, 'Overall']].dropna(); r, p = stats.spearmanr(d[col], d['Overall'])
    print(f'  {col:8s} rho = {r:.2f}, p = {p:.4f}')
print('\nProfile:'); print(f['Gender'].value_counts(normalize=True).round(2).to_string())
print(f['Age'].map({1: '1-12', 2: '13-30', 3: '31-60', 4:
```

```
'60+'}).value_counts(normalize=True).round(2).to_string())
media =
f['Hear_About'].dropna().astype(str).str.split(',').explode().str.strip().map({'1': 'TV',
'2': 'Newspaper', '3': 'Radio', '4': 'Website', '5': 'Word of mouth'})
out = media.value_counts().rename('mentions')
out.plot.bar(rot=0, title='How patrons heard about the series', color='#2E8B8B')
```

RESULT ON THE REAL CASE DATA

```
Overall satisfaction (%):
Overall
Good         48.8
Excellent    46.0
Average       3.7
Very poor     1.2
Poor          0.3

Spearman correlation of each factor with Overall (lower code = better):
  Sinage   rho = 0.46, p = 0.0000
  Parking  rho = 0.54, p = 0.0000
  Clean    rho = 0.43, p = 0.0000

Profile:
Gender
Female    0.65
Male      0.35
Age
13-30    0.53
31-60    0.36
1-12     0.08
60+      0.03
```

Hear_About	mentions
Word of mouth	232
Website	45
TV	23
Radio	17
Newspaper	14

Insight Hear_About allows more than one answer (for example '2,5'), so split those cells before counting. Word of mouth leads by a wide margin, followed by the website, so referral offers and the website are the strongest places to promote the series.

Python End-to-End Case Study: Marketing Analytics for an E-commerce Marketplace

A leading Indian online marketplace wants data-driven insight on business performance: customers, sellers, products, channels, payments and satisfaction. Clean the data, then answer the sample questions below and add your own analyses.

Data

Sep 2016–Oct 2018. Tables: CUSTOMERS, SELLERS, PRODUCTS, ORDERS, ORDER_ITEMS, ORDER_PAYMENTS, ORDER_REVIEW_RATINGS, GEO_LOCATION (~100k orders).

Tool and module

Python (pandas, matplotlib)
Python for Data Science

1a High-level metrics: revenue, quantity, products, categories, sellers, locations, payment methods.

PYTHON

```
out = pd.Series({
  'Total revenue (item price)': round(df['revenue'].sum(), 2),
  'Total quantity (items sold)': len(df),
  'Total orders': df['order_id'].nunique(),
  'Unique customers': df['customer_unique_id'].nunique(),
  'Total products': prod['product_id'].nunique(),
  'Total categories': prod['product_category_name'].nunique(),
  'Total sellers': sel['seller_id'].nunique(),
  'Customer states': cust['customer_state'].nunique(),
  'Customer cities': cust['customer_city'].nunique(),
  'Payment methods': pay['payment_type'].nunique(),
  'Average order value': round(df.groupby('order_id')['revenue'].sum().mean(), 2)},
name='value')
```

RESULT ON THE REAL CASE DATA

	value
Total revenue (item price)	13,591,643.70
Total quantity (items sold)	112,650.00
Total orders	98,666.00
Unique customers	95,420.00
Total products	32,951.00

	value
Total categories	72.00
Total sellers	3,095.00
Customer states	20.00
Customer cities	4,119.00
Payment methods	5.00
Average order value	137.75

1b New customers acquired every month.

PYTHON

```
first = df.groupby('customer_unique_id')
['order_purchase_timestamp'].min().dt.to_period('M')
nc = first.value_counts().sort_index()
nc.index = nc.index.to_timestamp(); nc.plot(marker='o', title='New customers per month');
out = nc.rename('new_customers')
```

RESULT ON THE REAL CASE DATA

order_purchase_timestamp	new_customers
2016-09-01	3
2016-10-01	305
2016-12-01	1
2017-01-01	754
2017-02-01	1705
2017-03-01	2595
2017-04-01	2339
2017-05-01	3560
2017-06-01	3114
2017-07-01	3843
2017-08-01	4149
2017-09-01	4090
2017-10-01	4412

order_purchase_timestamp	new_customers
2017-11-01	7217
2017-12-01	5442

Showing first 15 of 23 rows.

1c Month-on-month customer retention.

PYTHON

```
act = df[['customer_unique_id', 'month']].drop_duplicates()
act['cohort'] = act['customer_unique_id'].map(first)
ret = act[act['month'] > act['cohort']].groupby('month')['customer_unique_id'].nunique()
tot = act.groupby('month')['customer_unique_id'].nunique()
out = pd.DataFrame({'active': tot, 'returning': ret}).fillna(0).astype(int)
out['returning_%'] = (out['returning'] / out['active'] * 100).round(2)
print(f"Answer: only {act.groupby('customer_unique_id').size().gt(1).mean()*100:.1f}% of
customers ever bought in more than one month, so retention is very low.")
```

RESULT ON THE REAL CASE DATA

```
Answer: only 1.8% of customers ever bought in more than one month, so retention is very
low.
```

month	active	returning	returning_%
2016-09	3	0	0.00
2016-10	305	0	0.00
2016-12	1	0	0.00
2017-01	755	1	0.13
2017-02	1708	3	0.18
2017-03	2601	6	0.23
2017-04	2359	20	0.85
2017-05	3588	28	0.78
2017-06	3154	40	1.27

month	active	returning	returning_%
2017-07	3894	51	1.31
2017-08	4211	62	1.47
2017-09	4170	80	1.92
2017-10	4501	89	1.98
2017-11	7342	125	1.70
2017-12	5557	115	2.07

Showing first 15 of 24 rows.

1d Revenue from new vs existing customers by month.

PYTHON

```
df['cohort'] = df['customer_unique_id'].map(first)
df['cust_type'] = np.where(df['month'] == df['cohort'], 'New', 'Existing')
nv = df.pivot_table(index='month', columns='cust_type', values='revenue',
aggfunc='sum').fillna(0)
nv.index = nv.index.to_timestamp(); nv.plot.area(title='Revenue: new vs existing
customers'); out = nv.round(0)
```

RESULT ON THE REAL CASE DATA

month	cust_type Existing	New
2016-09-01	0.00	267.00
2016-10-01	0.00	49,508.00
2016-12-01	0.00	11.00
2017-01-01	11.00	120,302.00
2017-02-01	125.00	247,178.00
2017-03-01	561.00	373,783.00
2017-04-01	2,153.00	357,774.00
2017-05-01	4,380.00	501,691.00
2017-06-01	5,028.00	428,010.00
2017-07-01	8,562.00	489,469.00

cust_type	Existing	New
month		
2017-08-01	9,539.00	564,433.00
2017-09-01	10,116.00	614,285.00
2017-10-01	11,475.00	652,744.00
2017-11-01	15,237.00	995,034.00
2017-12-01	16,443.00	727,471.00

Showing first 15 of 24 rows.

1e Trends and seasonality: by month, weekday, hour, and payment method.

PYTHON

```
fig, ax = plt.subplots(1, 3, figsize=(14, 3.6))
m = df.groupby('month')['revenue'].sum(); m.index = m.index.to_timestamp();
m.plot(ax=ax[0], title='Revenue by month')
df.groupby(df['order_purchase_timestamp'].dt.day_name())
['revenue'].sum().reindex(['Monday','Tuesday','Wednesday','Thursday','Friday','Saturday','
Sunday']).plot.bar(ax=ax[1], title='Revenue by weekday', rot=45)
df.groupby(df['order_purchase_timestamp'].dt.hour)
['order_id'].nunique().plot.bar(ax=ax[2], title='Orders by hour of day', rot=0)
```

RESULT ON THE REAL CASE DATA

1f Popular products by month, seller, state and category (top 10 products overall shown).

PYTHON

```
out = df.groupby(['product_id',
'product_category_name']).size().sort_values(ascending=False).head(10).rename('units')
```

RESULT ON THE REAL CASE DATA

		units
product_id	product_category_name	
aca2eb7d00ea1a7b8ebd4e68314663af	Furniture_Decor	527

product_id	product_category_name	units
99a4788cb24856965c36a24e339b6058	Bed_Bath_Table	488
422879e10f46682990de24d770e7f83d	Garden_Tools	484
389d119b48cf3043d311335e499d9c6b	Garden_Tools	392
368c6c730842d78016ad823897a372db	Garden_Tools	388
53759a2ecddad2bb87a079a1f1519f73	Garden_Tools	373
d1c427060a0f73f6b889a5c7c61f2ac4	Computers_Accessories	343
53b36df67ebb7c41585e8d54d6772e08	Watches_Gifts	323
154e7e31ebfa092203795c972e5804a6	Health_Beauty	281
3dd2a17168ec895c781a9191c1e95ad7	Computers_Accessories	274

1g Popular categories by state.

PYTHON

```
top = df.groupby(['customer_state',
'product_category_name']).size().rename('units').reset_index()
out = top.sort_values('units',
ascending=False).drop_duplicates('customer_state').set_index('customer_state').head(15)
```

RESULT ON THE REAL CASE DATA

customer_state	product_category_name	units
Andhra Pradesh	Bed_Bath_Table	7101
Gujarat	Bed_Bath_Table	786
Chhattisgarh	Bed_Bath_Table	681
Karnataka	Bed_Bath_Table	379
Delhi	Health_Beauty	328
Madhya Pradesh	Bed_Bath_Table	319
Haryana	Bed_Bath_Table	282
Maharashtra	Bed_Bath_Table	213
Tamil Nadu	Bed_Bath_Table	212

customer_state	product_category_name	units
Jammu & Kashmir	Bed_Bath_Table	160
Kerala	Bed_Bath_Table	146
Arunachal Pradesh	Bed_Bath_Table	114
Uttar Pradesh	Health_Beauty	104
Rajasthan	Sports_Leisure	97
Himachal Pradesh	Bed_Bath_Table	83

1h Top 10 most expensive products sorted by price.

PYTHON

```
out = items.groupby('product_id')
['price'].max().sort_values(ascending=False).head(10).to_frame().merge(prod[['product_id',
'product_category_name']].set_index('product_id'), left_index=True, right_index=True)
```

RESULT ON THE REAL CASE DATA

product_id	price	product_category_name
489ae2aa008f021502940f251d4cce7f	6,735.00	Housewares
69c590f7ffc7bf8db97190b6cb6ed62e	6,729.00	Computers
1bdf5e6731585cf01aa8169c7028d6ad	6,499.00	Art
a6492cc69376c469ab6f61d8f44de961	4,799.00	Small_Appliances
c3ed642d592594bb648ff4a04cee2747	4,690.00	Small_Appliances
259037a6a41845e455183f89c5035f18	4,590.00	Computers
a1beef8f3992dbd4cd8726796aa69c53	4,399.87	Musical_Instruments
6cdf8fc1d741c76586d8b6b15e9eef30	4,099.99	Consoles_Games
dd113cb02b2af9c8e5787e8f1f0722f6	4,059.00	Sports_Leisure
6902c1962dd19d540807d0ab8fade5c6	3,999.90	Watches_Gifts

2a Segment customers into groups by revenue generated.

PYTHON

```python
cr = df.groupby('customer_unique_id')['revenue'].sum()
seg = pd.qcut(cr, 4, labels=['Low', 'Medium', 'High', 'Premium'])
out = cr.groupby(seg, observed=True).agg(['count', 'sum', 'mean']).round(2)
```

RESULT ON THE REAL CASE DATA

revenue	count	sum	mean
Low	23984	701,218.38	29.24
Medium	24869	1,660,701.82	66.78
High	22731	2,716,863.83	119.52
Premium	23836	8,512,859.67	357.14

2b Segment sellers into groups by revenue generated.

PYTHON

```python
sr = df.groupby('seller_id')['revenue'].sum()
out = sr.groupby(pd.qcut(sr, 4, labels=['Low', 'Medium', 'High', 'Top']),
observed=True).agg(['count', 'sum', 'mean']).round(2)
print(f"Top 25% of sellers bring {out.loc['Top','sum']/sr.sum()*100:.1f}% of revenue.")
```

RESULT ON THE REAL CASE DATA

```
Top 25% of sellers bring 87.1% of revenue.
```

revenue	count	sum	mean
Low	774	78,423.16	101.32
Medium	774	359,264.39	464.17
High	773	1,315,369.95	1,701.64
Top	774	11,838,586.20	15,295.33

3 Cross-selling: top 10 category combinations bought together in the same order.

PYTHON

```python
from itertools import combinations
from collections import Counter
cnt = Counter()
```

```
for cats in df.groupby('order_id')['product_category_name'].apply(lambda s:
sorted(set(s))):
    for k in (2, 3):
        cnt.update(combinations(cats, k))
out = pd.Series({' + '.join(k): v for k, v in cnt.most_common(10)}, name='orders')
print('Orders with 2+ different categories:', (df.groupby('order_id')
['product_category_name'].nunique() > 1).sum())
```

RESULT ON THE REAL CASE DATA

```
Orders with 2+ different categories: 786
```

	orders
Bed_Bath_Table + Furniture_Decor	70
Bed_Bath_Table + Home_Confort	43
Furniture_Decor + Housewares	24
Bed_Bath_Table + Housewares	20
Baby + Cool_Stuff	20
Baby + Toys	19
Furniture_Decor + Garden_Tools	17
Baby + Bed_Bath_Table	17
Housewares + Unknown	14
Health_Beauty + Sports_Leisure	14

Insight Cross-selling is analysed at category level. At product level almost no combination repeats, because most orders contain a single product.

4 Payment behaviour: how do customers pay, and which channel is used most?

PYTHON

```
p = pay.groupby('payment_type').agg(orders=('order_id', 'nunique'), value=
('payment_value', 'sum'), avg_instalments=('payment_installments',
'mean')).sort_values('orders', ascending=False)
p['share_%'] = (p['orders'] / p['orders'].sum() * 100).round(1); out = p.round(2)
p['orders'].plot.bar(rot=0, title='Orders by payment method')
```

RESULT ON THE REAL CASE DATA

payment_type	orders	value	avg_instalments	share_%
credit_card	76505	12,542,084.19	3.51	75.20
UPI	19784	2,869,361.27	1.00	19.50
voucher	3866	379,436.87	1.00	3.80
debit_card	1528	217,989.79	1.00	1.50
not_defined	3	0.00	1.00	0.00

5a Top 10 and bottom 10 categories by average rating.

PYTHON

```
r = df.merge(rev[['order_id', 'review_score']], on='order_id')
cat = r.groupby('product_category_name')['review_score'].agg(['count', 'mean'])
cat = cat[cat['count'] >= 30]
print('TOP 10:'); print(cat.sort_values('mean',
ascending=False).head(10).round(2).to_string())
print('\nBOTTOM 10:'); print(cat.sort_values('mean').head(10).round(2).to_string())
```

RESULT ON THE REAL CASE DATA

```
TOP 10:
                                   count  mean
product_category_name
Books_General_Interest               553  4.44
Books_Imported                        60  4.40
Costruction_Tools_Tools              103  4.36
Books_Technical                      269  4.34
Luggage_Accessories                 1092  4.31
Food_Drink                           280  4.30
Small_Appliances_Home_Oven_And_Coffee 76  4.30
Fashion_Sport                         31  4.26
Flowers                               33  4.21
Music                                 38  4.21

BOTTOM 10:
                                 count  mean
product_category_name
Diapers_And_Hygiene                 39  3.26
Home_Comfort_2                      30  3.37
Office_Furniture                  1701  3.48
Fashion_Male_Clothing              132  3.62
Fixed_Telephony                    265  3.67
Party_Supplies                      43  3.77
Fashio_Female_Clothing              50  3.78
```

```
Audio                                365  3.81
Unknown                             1636  3.81
Furniture_Mattress_And_Upholstery     38  3.82
```

Insight Categories with fewer than 30 reviews are excluded so that one or two ratings cannot put a category at the top or bottom.

5c Average rating by state and by month; link to delivery delay.

PYTHON

```
r['late'] = r['order_delivered_customer_date'] > r['order_estimated_delivery_date']
print('Average rating, on time vs late:'); print(r.groupby('late')
['review_score'].mean().round(2).to_string())
m = r.groupby('month')['review_score'].mean(); m.index = m.index.to_timestamp()
m.plot(marker='o', title='Average rating by month'); out = r.groupby('customer_state')
['review_score'].mean().sort_values().round(2).head(10)
```

RESULT ON THE REAL CASE DATA

```
Average rating, on time vs late:
late
False    4.14
True     2.53
```

customer_state	review_score
Uttaranchal	3.70
Andhra Pradesh	4.00
Himachal Pradesh	4.00
West Bengal	4.00
Madhya Pradesh	4.01
Karnataka	4.01
Jammu & Kashmir	4.02
Chhattisgarh	4.03
Gujarat	4.03
Delhi	4.03

EXCEL (DATA VISUALIZATION & ANALYTICS) · RESOLUTION

Excel Case Study: Sportsmen Roster (Data Cleaning, Analysis, Report)

XYZ Co organises international sports events. Countries nominate sportsmen, and the team must standardise the membership roster (Stage 1), summarise it (Stage 2) and build a filterable report (Stage 3).

Data

Sheets: SPORTSMEN (50 members, 19 columns), SPORT (sport → indoor/outdoor), LOCATION (country code → country name and language, laid out horizontally).

Tool and module

Microsoft Excel
Data Visualization & Analytics (Excel)

1.1 Populate FULL NAME as PREFIX FIRSTNAME LASTNAME in UPPER CASE (column B).

EXCEL STEPS / FORMULAS

```
B2:  =UPPER(C2&" "&D2&" "&F2)
     (or =UPPER(TEXTJOIN(" ",TRUE,C2,D2,F2)) ; fill down)
```

EXPECTED RESULT (COMPUTED FROM THE CASE DATA, USE IT TO CHECK YOUR WORK)

	MEMBER ID	FULL NAME
0	1	MS. ANNIE ABBOTT
1	2	MS. AURELIE LIESUCHKE
2	3	SR. TOMAS FILHO
3	4	MS. DARBY CRUICKSHANK
4	5	DR. JAYDON BORER

1.2 Get COUNTRY NAME from the LOCATION sheet (column K).

EXCEL STEPS / FORMULAS

```
K2:  =HLOOKUP(J2, LOCATION!$B$2:$M$3, 2, FALSE)
     LOCATION is laid out horizontally (row 1 = language, row 2 = code, row 3 = country),
so use HLOOKUP,
     or =INDEX(LOCATION!$B$3:$M$3, MATCH(J2, LOCATION!$B$2:$M$2, 0))
```

EXPECTED RESULT (COMPUTED FROM THE CASE DATA, USE IT TO CHECK YOUR WORK)

	COUNTRYCODE	COUNTRY NAME
0	US	USA
2	BR	BRAZIL
8	GB	UK
13	DE	GERMANY
18	OZ	AUSTRALIA
26	AU	AUSTRIA
29	FR	FRANCE
38	AG	ARGENTINA
41	ES	SPAIN
44	DU	NETHERLANDS
47	SV	SWEDEN
49	PR	BRAZIL

1.3 Populate LANGUAGE from the LOCATION sheet (column L).

EXCEL STEPS / FORMULAS

```
L2:  =INDEX(LOCATION!$B$1:$M$1, MATCH(J2, LOCATION!$B$2:$M$2, 0))
      (the language row sits ABOVE the lookup row, so HLOOKUP cannot reach it; INDEX/MATCH can)
```

EXPECTED RESULT (COMPUTED FROM THE CASE DATA, USE IT TO CHECK YOUR WORK)

	COUNTRYCODE	LANGUAGE
0	US	English
2	BR	Portuguese
8	GB	English
13	DE	German
18	OZ	English
26	AU	German
29	FR	French
38	AG	Spanish

	COUNTRYCODE	LANGUAGE
41	ES	Spanish
44	DU	Dutch
47	SV	Swedish
49	PR	Portuguese

1.4 EMAIL: lastname.firstname@xyz.org for English speakers, @xyz.com for others, all lower case (column M).

EXCEL STEPS / FORMULAS

```
M2:  =LOWER(F2&"."&D2) & IF(L2="English","@xyz.org","@xyz.com")
```

EXPECTED RESULT (COMPUTED FROM THE CASE DATA, USE IT TO CHECK YOUR WORK)

	FIRSTNAME	LASTNAME	LANGUAGE	EMAIL
0	Annie	Abbott	English	abbott.annie@xyz.org
1	Aurelie	Liesuchke	English	liesuchke.aurelie@xyz.org
2	Tomas	Filho	Portuguese	filho.tomas@xyz.com
3	Darby	Cruickshank	English	cruickshank.darby@xyz.org
4	Jaydon	Borer	English	borer.jaydon@xyz.org
5	Moriah	Lynch	English	lynch.moriah@xyz.org
6	Amiya	Eichmann	English	eichmann.amiya@xyz.org
7	Pierce	Rau	English	rau.pierce@xyz.org

1.5 Populate SPORT LOCATION (indoor/outdoor) from the SPORT sheet (column Q).

EXCEL STEPS / FORMULAS

```
Q2:  =INDEX(SPORT!$A:$A, MATCH(R2, SPORT!$B:$B, 0))
      (the result column is LEFT of the lookup column, so VLOOKUP needs CHOOSE:
=VLOOKUP(R2, CHOOSE({1,2}, SPORT!$B:$B, SPORT!$A:$A), 2, 0) or use XLOOKUP)
```

EXPECTED RESULT (COMPUTED FROM THE CASE DATA, USE IT TO CHECK YOUR WORK)

SPORT LOCATION	count
OUTDOOR	32
INDOOR	18

2.1 Show MEMBER ID as a 3-digit number (001, 002 ...).

EXCEL STEPS / FORMULAS

```
Select column A → Ctrl+1 → Custom → Type: 000
(keeps the value numeric; =TEXT(A2,"000") would convert it to text)
```

EXPECTED RESULT (COMPUTED FROM THE CASE DATA, USE IT TO CHECK YOUR WORK)

	MEMBER ID
0	001
1	002
2	003
3	004
4	005

2.2 Format BIRTHDATE as dd mmm' yyyy (e.g. 09 May' 1986).

EXCEL STEPS / FORMULAS

```
Column G → Ctrl+1 → Custom → Type:  dd mmm' yyyy
```

EXPECTED RESULT (COMPUTED FROM THE CASE DATA, USE IT TO CHECK YOUR WORK)

	BIRTHDATE
0	26 Sep' 1997
1	07 Feb' 1992
2	10 Jul' 1969
3	18 May' 1975
4	18 May' 1970

2.3 Show WEIGHT with units (e.g. 80 kg).

EXCEL STEPS / FORMULAS

```
Column N → Ctrl+1 → Custom → Type:  0.0 "kg"   (or  General" kg")
```

EXPECTED RESULT (COMPUTED FROM THE CASE DATA, USE IT TO CHECK YOUR WORK)

	WEIGHT
0	94 kg
1	84.2 kg
2	52.9 kg
3	48.9 kg
4	84.8 kg

2.4 SALARY in thousands: 2 decimals below 100,000, 1 decimal otherwise (87670 → 87.67 k; 123250 → 123.2 k).

EXCEL STEPS / FORMULAS

```
Column S → Ctrl+1 → Custom → Type:
[<100000]0.00, "k";0.0, "k"
(each comma after the number divides by 1,000; the [condition] section picks the decimals)
```

EXPECTED RESULT (COMPUTED FROM THE CASE DATA, USE IT TO CHECK YOUR WORK)

	SALARY
0	80.73 k
1	87.47 k
2	64.72 k
3	110.8 k
4	56.92 k
5	51.13 k
6	65.47 k
7	109.9 k

Stage 2 Pivot (ANALYSIS!B3): COUNTRY in rows, GENDER in columns, count of members, no grand totals; plus the same cross-table with functions at G4.

EXCEL STEPS / FORMULAS

```
Pivot: Insert → PivotTable → ANALYSIS!B3 → Rows: COUNTRY NAME, Columns: GENDER, Values:
Count of MEMBER ID
       Design → Grand Totals → Off for Rows and Columns
Functions: H4:I4  copy GENDER → Data → Remove Duplicates → Paste Special → Transpose
           G5:G16 copy COUNTRY NAME → Remove Duplicates
           H5:   =COUNTIFS(SPORTSMEN!$K:$K, $G5, SPORTSMEN!$I:$I, H$4)   → fill across and
down
```

EXPECTED RESULT (COMPUTED FROM THE CASE DATA, USE IT TO CHECK YOUR WORK)

GENDER	Female	Male
COUNTRY NAME		
ARGENTINA	1	2
AUSTRALIA	6	2
AUSTRIA	1	2
BRAZIL	0	2
FRANCE	3	6
GERMANY	1	4
NETHERLANDS	2	1
SPAIN	3	0
SWEDEN	1	1
UK	3	2
USA	4	3

Stage 3 Pivot report (REPORT!A3) with member details in tabular form, years only for birth date, no +/- buttons or grand totals, filter by SPORT LOCATION.

EXCEL STEPS / FORMULAS

```
Insert → PivotTable → REPORT!A3
Rows (in order): MEMBER ID, FULL NAME, EMAIL, GENDER, BIRTHDATE, COUNTRY NAME, LANGUAGE,
SPORTS
Right-click BIRTHDATE → Group → Years only (untick Months, Quarters, Days)
Filters area: SPORT LOCATION (appears at A1)
Design → Report Layout → Show in Tabular Form; Subtotals → Do Not Show; Grand Totals → Off
PivotTable Analyze → +/- Buttons (toggle off)
```

EXPECTED RESULT (COMPUTED FROM THE CASE DATA, USE IT TO CHECK YOUR WORK)

	MEMBER ID	FULL NAME	EMAIL	GENDER	COUNTRY NAME	LANGUAGE
0	1	MS. ANNIE ABBOTT	abbott.annie@xyz.org	Female	USA	English
1	2	MS. AURELIE LIESUCHKE	liesuchke.aurelie@xyz.org	Female	USA	English
2	3	SR. TOMAS FILHO	filho.tomas@xyz.com	Male	BRAZIL	Portuguese
3	4	MS. DARBY CRUICKSHANK	cruickshank.darby@xyz.org	Female	USA	English
4	5	DR. JAYDON BORER	borer.jaydon@xyz.org	Male	USA	English
5	6	MR. MORIAH LYNCH	lynch.moriah@xyz.org	Male	USA	English
6	7	MS. AMIYA EICHMANN	eichmann.amiya@xyz.org	Female	USA	English
7	8	MR. PIERCE RAU	rau.pierce@xyz.org	Male	USA	English

EXCEL (DATA VISUALIZATION & ANALYTICS) · RESOLUTION

Excel Case Study: Consumer Complaint Analysis

Analyse US consumer financial complaints: map state names from JSON, fix dates, report at company level, find the top companies and issues, and build a year-level KPI dashboard with priorities.

Data

Consumer_Complaints (one row per complaint: company, product, issue, state, channel, received/resolved dates, timely response, disputed) · State_Code_Name (JSON text per state).

Tool and module

Microsoft Excel
Data Visualization & Analytics (Excel)

T1.1–1.2 Split the JSON text in State_Code_Name into STATE and CODE columns.

EXCEL STEPS / FORMULAS

```
STATE: =MID(A2, FIND(":",A2)+3, FIND(",",A2)-FIND(":",A2)-4)
CODE:  =MID(A2, FIND("CODE",A2)+8, 2)
(or Data → From Table/Range → Power Query → Transform → Parse → JSON → Expand)
```

EXPECTED RESULT (COMPUTED FROM THE CASE DATA, USE IT TO CHECK YOUR WORK)

	STATE	CODE
0	Alabama	AL
1	Alaska	AK
2	Arizona	AZ
3	Arkansas	AR
4	California	CA

T1.3–1.5 Add State_Name to Consumer_Complaints by lookup, and count the #N/A records.

EXCEL STEPS / FORMULAS

```
State_Name:  =VLOOKUP(E2, CHOOSE({1,2}, State_Code_Name!$C:$C, State_Code_Name!$B:$B), 2,
FALSE)
            or =XLOOKUP(E2, State_Code_Name!$C:$C, State_Code_Name!$B:$B)
Count of #N/A: =SUMPRODUCT(--ISNA(K2:K20000))   or  =COUNTIF(K:K,"#N/A")
```

EXPECTED RESULT (COMPUTED FROM THE CASE DATA, USE IT TO CHECK YOUR WORK)

```
Answer: records with #N/A = 157
```

T1.6–1.7 Complaints by State code where State_Name did not map, and why.

EXCEL STEPS / FORMULAS

```
Pivot on rows filtered to State_Name = #N/A → Rows: State, Values: Count of Complaint ID
```

EXPECTED RESULT (COMPUTED FROM THE CASE DATA, USE IT TO CHECK YOUR WORK)

State	complaints
(blank)	110
PR	26
VI	8
AE	5
AP	4
AS	1
GU	1
MP	1
AA	1

Insight The unmatched codes are US territories and military mail codes that are not among the 51 states in the lookup list (for example PR, AE, AP, GU, VI), plus blank state values. Add these codes to the lookup sheet, or report them as 'Other/Unknown'.

T2 Resolution time in days, YEAR column, and US-FY quarter (Q1 = Jan–Mar).

EXCEL STEPS / FORMULAS

```
Fix text dates (mm/dd/yyyy): =DATE(RIGHT(G2,4), LEFT(G2,2), MID(G2,4,2))
Fix true dates that were read as dd/mm: =DATE(YEAR(G2), DAY(G2), MONTH(G2))
Combined: =IF(ISTEXT(G2), DATE(RIGHT(G2,4),LEFT(G2,2),MID(G2,4,2)),
DATE(YEAR(G2),DAY(G2),MONTH(G2)))
Resolution days: =Resolved - Received
YEAR: =YEAR(Received)      QTR: ="Q"&ROUNDUP(MONTH(Received)/3,0)
```

EXPECTED RESULT (COMPUTED FROM THE CASE DATA, USE IT TO CHECK YOUR WORK)

```
Negative resolution times after fixing: 3
```

	Received	Resolved	Resolution_days	YEAR	QTR
0	2015-10-20	2015-10-20	0	2015	Q4
1	2014-01-03	2014-01-03	0	2014	Q1
2	2014-12-04	2014-12-04	0	2014	Q4
3	2015-03-26	2015-03-26	0	2015	Q1
4	2015-10-27	2015-10-27	0	2015	Q4

T3 Company-level report: complaints (desc), number and % not timely, number and % disputed, average delay.

EXCEL STEPS / FORMULAS

```
Pivot (Report_Task3): Rows = Company; Values = Count of Complaint ID (sort desc)
Add helper columns: NotTimely =--(I2="No"), Disputed =--(J2="Yes")
Values: Sum of NotTimely, Sum of Disputed, Average of Resolution_days
% columns: next to the pivot,  =Sum of NotTimely / Count of Complaint ID  (format as %),
same for Disputed
```

EXPECTED RESULT (COMPUTED FROM THE CASE DATA, USE IT TO CHECK YOUR WORK)

```
Companies: 1050
```

Company	Complaints	Not_timely	Disputed	Avg_delay_days	Not_timely_%	Disputed_%
Bank of America	1066	5	248	1.90	0.50	23.30
Wells Fargo & Company	947	21	221	1.70	2.20	23.30
JPMorgan Chase & Co.	877	1	239	1.70	0.10	27.30
Citibank	764	1	152	1.50	0.10	19.90
Equifax	531	0	133	1.00	0.00	25.00
Capital One	458	3	72	1.60	0.70	15.70
Experian	456	0	59	1.40	0.00	12.90
Ocwen	437	11	126	1.40	2.50	28.80

Company	Complaints	Not_timely	Disputed	Avg_delay_days	Not_timely_%	Disputed_%
Navient Solutions, Inc.	350	0	75	1.20	0.00	21.40
U.S. Bancorp	309	1	63	1.70	0.30	20.40
TransUnion Intermediate Holdings, Inc.	302	0	66	1.40	0.00	21.90
Nationstar Mortgage	286	0	82	1.40	0.00	28.70
Synchrony Financial	284	0	45	2.10	0.00	15.80
Ditech Financial LLC	254	1	57	1.10	0.40	22.40
Amex	195	0	58	1.20	0.00	29.70

Showing first 15 of 1,050 rows.

T4 Top 5 companies and top 5 issues by complaints; monthly trend chart.

EXCEL STEPS / FORMULAS

```
Pivot → Rows: Company → Value Filters → Top 10 → change to Top 5 by Count
Same for Issue
Pivot → Rows: Received grouped by Years + Months → Insert → Line/Area chart
```

EXPECTED RESULT (COMPUTED FROM THE CASE DATA, USE IT TO CHECK YOUR WORK)

```
Top 5 companies:
Company
Bank of America          1066
Wells Fargo & Company     947
JPMorgan Chase & Co.      877
Citibank                  764
Equifax                   531

Top 5 issues:
Issue
Loan servicing, payments, escrow account    2354
Account opening, closing, or management     1047
Loan modification,collection,foreclosure     883
```

```
Communication tactics                    814
Deposits and withdrawals                 618
```

T5 Year-level KPIs with YoY change; product and channel mix; priority view.

EXCEL STEPS / FORMULAS

```
KPIs: pivot with Rows = YEAR, Values = Count, Sum of Timely flag, Average Resolution_days
YoY %: =(ThisYear - LastYear) / LastYear   (or Show Values As → % Difference From → Previous)
Product and channel mix: pie/donut on pivot counts; Priority: scatter of complaints vs % disputed by Issue
```

EXPECTED RESULT (COMPUTED FROM THE CASE DATA, USE IT TO CHECK YOUR WORK)

YEAR	Complaints	Timely	Avg_resolution	Complaints_YoY_%	Timely_YoY_%	Avg_resoluti
2013	2497	2486	2.62	NaN	NaN	NaN
2014	3472	3424	2.19	39.00	37.70	-16.40
2015	4373	4269	1.85	26.00	24.70	-15.30
2016	3658	3596	1.60	-16.40	-15.80	-13.90

Insight For the priority view, plot each issue's complaint volume against its dispute rate. Issues that are high on both axes are the ones to handle first with limited bandwidth.

SQL (DATA VISUALIZATION & ANALYTICS) · RESOLUTION

SQL Case Study (Basic): Retail Data Analysis

A retail store wants to understand customer behaviour from its point-of-sale data. Write one SQL query per question, assuming read-only access.

Data

Customer (5,647 rows) · Transactions (23,053 rows; negative Qty/amount = return) · prod_cat_info (23 rows). Join Transactions to prod_cat_info on BOTH prod_cat_code and prod_subcat_code.

Tool and module

SQL Server (T-SQL)

Data Visualization & Analytics (SQL)

DP1 Total number of rows in each of the 3 tables.

T-SQL (SQL SERVER)

```sql
SELECT 'Customer' AS table_name, COUNT(*) AS row_count FROM Customer
UNION ALL SELECT 'Transactions', COUNT(*) FROM Transactions
UNION ALL SELECT 'prod_cat_info', COUNT(*) FROM prod_cat_info;
```

RESULT ON THE REAL CASE DATA

	table_name	row_count
0	Customer	5647
1	Transactions	23053
2	prod_cat_info	23

DP2 Total number of transactions that have a return.

T-SQL (SQL SERVER)

```sql
SELECT COUNT(DISTINCT transaction_id) AS return_transactions
FROM Transactions
WHERE Qty < 0;
```

RESULT ON THE REAL CASE DATA

	return_transactions
0	2059

DP3 Convert the date columns to valid date formats.

T-SQL (SQL SERVER)

```
-- Dates arrive as text in dd-mm-yyyy. Style 105 = dd-mm-yyyy
SELECT transaction_id, TRY_CONVERT(DATE, tran_date, 105) AS tran_date
FROM Transactions;
-- Permanent fix when you own the table:
-- ALTER TABLE Transactions ALTER COLUMN tran_date DATE;
```

RESULT ON THE REAL CASE DATA

	transaction_id	tran_date
0	80712190438	2014-02-28
1	29258453508	2014-02-27
2	51750724947	2014-02-24
3	93274880719	2014-02-24
4	51750724947	2014-02-23

DP4 Time range of the transaction data, in days, months and years (separate columns).

T-SQL (SQL SERVER)

```
SELECT DATEDIFF(DAY,   MIN(tran_date), MAX(tran_date)) AS days_range,
       DATEDIFF(MONTH, MIN(tran_date), MAX(tran_date)) AS months_range,
       DATEDIFF(YEAR,  MIN(tran_date), MAX(tran_date)) AS years_range
FROM Transactions;
```

RESULT ON THE REAL CASE DATA

	days_range	months_range	years_range
0	1130	37	3

DP5 Which product category does the sub-category 'DIY' belong to?

T-SQL (SQL SERVER)

```
SELECT prod_cat
FROM prod_cat_info
WHERE prod_subcat = 'DIY';
```

RESULT ON THE REAL CASE DATA

	prod_cat
0	Books

DA1 Which channel (store type) is most frequently used for transactions?

T-SQL (SQL SERVER)

```
SELECT TOP 1 Store_type, COUNT(transaction_id) AS txn_count
FROM Transactions
GROUP BY Store_type
ORDER BY txn_count DESC;
```

RESULT ON THE REAL CASE DATA

	Store_type	txn_count
0	e-Shop	9311

DA2 Count of male and female customers.

T-SQL (SQL SERVER)

```
SELECT Gender, COUNT(customer_Id) AS customers
FROM Customer
WHERE Gender IS NOT NULL
GROUP BY Gender;
```

RESULT ON THE REAL CASE DATA

	Gender	customers
0	M	2892
1	F	2753

DA3 City with the maximum number of customers, and how many.

T-SQL (SQL SERVER)

```
SELECT TOP 1 city_code, COUNT(customer_Id) AS customers
FROM Customer
GROUP BY city_code
ORDER BY customers DESC;
```

RESULT ON THE REAL CASE DATA

	city_code	customers
0	3.00	595

DA4 How many sub-categories are there under Books?

T-SQL (SQL SERVER)

```
SELECT COUNT(prod_subcat) AS subcategories
FROM prod_cat_info
WHERE prod_cat = 'Books';
```

RESULT ON THE REAL CASE DATA

	subcategories
0	6

DA5 Maximum quantity of products ever ordered.

T-SQL (SQL SERVER)

```
SELECT MAX(Qty) AS max_qty
FROM Transactions;
```

RESULT ON THE REAL CASE DATA

	max_qty
0	5

DA6 Net total revenue in Electronics and Books.

T-SQL (SQL SERVER)

```
SELECT ROUND(SUM(t.total_amt), 2) AS net_revenue
FROM Transactions t
JOIN prod_cat_info p
  ON t.prod_cat_code = p.prod_cat_code AND t.prod_subcat_code = p.prod_sub_cat_code
WHERE p.prod_cat IN ('Electronics', 'Books');
```

RESULT ON THE REAL CASE DATA

	net_revenue
0	23,545,157.67

DA7 How many customers have more than 10 transactions, excluding returns?

T-SQL (SQL SERVER)

```
SELECT COUNT(*) AS customers
FROM (
    SELECT cust_id
    FROM Transactions
    WHERE Qty > 0
    GROUP BY cust_id
    HAVING COUNT(DISTINCT transaction_id) > 10
) x;
```

RESULT ON THE REAL CASE DATA

	customers
0	6

DA8 Combined revenue from Electronics and Clothing at Flagship stores.

T-SQL (SQL SERVER)

```
SELECT ROUND(SUM(t.total_amt), 2) AS revenue
FROM Transactions t
JOIN prod_cat_info p
  ON t.prod_cat_code = p.prod_cat_code AND t.prod_subcat_code = p.prod_sub_cat_code
WHERE p.prod_cat IN ('Electronics', 'Clothing') AND t.Store_type = 'Flagship store';
```

RESULT ON THE REAL CASE DATA

	revenue
0	3,409,559.27

DA9 Total revenue from male customers in Electronics, by sub-category.

T-SQL (SQL SERVER)

```
SELECT p.prod_subcat, ROUND(SUM(t.total_amt), 2) AS revenue
FROM Transactions t
JOIN Customer c ON t.cust_id = c.customer_Id
JOIN prod_cat_info p
  ON t.prod_cat_code = p.prod_cat_code AND t.prod_subcat_code = p.prod_sub_cat_code
WHERE c.Gender = 'M' AND p.prod_cat = 'Electronics'
GROUP BY p.prod_subcat
ORDER BY revenue DESC;
```

RESULT ON THE REAL CASE DATA

	prod_subcat	revenue
0	Mobiles	1,192,413.23
1	Cameras	1,172,702.25
2	Audio and video	1,138,983.17
3	Personal Appliances	1,107,593.43
4	Computers	1,091,417.34

DA10 Percentage of sales and returns by sub-category; top 5 sub-categories by sales.

T-SQL (SQL SERVER)

```
SELECT TOP 5 p.prod_subcat,
       ROUND(100.0 * SUM(CASE WHEN t.total_amt > 0 THEN t.total_amt ELSE 0 END)
             / (SELECT SUM(total_amt) FROM Transactions WHERE total_amt > 0), 2) AS
sales_pct,
       ROUND(100.0 * SUM(CASE WHEN t.total_amt < 0 THEN t.total_amt ELSE 0 END)
             / (SELECT SUM(total_amt) FROM Transactions WHERE total_amt < 0), 2) AS
returns_pct
FROM Transactions t
JOIN prod_cat_info p
  ON t.prod_cat_code = p.prod_cat_code AND t.prod_subcat_code = p.prod_sub_cat_code
GROUP BY p.prod_subcat
ORDER BY sales_pct DESC;
```

RESULT ON THE REAL CASE DATA

	prod_subcat	sales_pct	returns_pct
0	Women	12.89	14.52
1	Mens	12.68	12.49
2	Kids	8.83	9.37
3	Mobiles	4.61	4.43
4	Fiction	4.58	4.44

DA11 Net revenue from customers aged 25–35 in the last 30 days of transactions (from the max transaction date).

T-SQL (SQL SERVER)

```
SELECT ROUND(SUM(t.total_amt), 2) AS net_revenue
FROM Transactions t
JOIN Customer c ON t.cust_id = c.customer_Id
```

```
WHERE DATEDIFF(YEAR, c.DOB, t.tran_date) BETWEEN 25 AND 35
  AND t.tran_date >= DATEADD(DAY, -30, (SELECT MAX(tran_date) FROM Transactions));
```

RESULT ON THE REAL CASE DATA

	net_revenue
0	448,137.17

DA12 Product category with the maximum value of returns in the last 3 months of transactions.

T-SQL (SQL SERVER)

```
SELECT TOP 1 p.prod_cat, ROUND(SUM(t.total_amt), 2) AS return_value
FROM Transactions t
JOIN prod_cat_info p
  ON t.prod_cat_code = p.prod_cat_code AND t.prod_subcat_code = p.prod_sub_cat_code
WHERE t.total_amt < 0
  AND t.tran_date >= DATEADD(MONTH, -3, (SELECT MAX(tran_date) FROM Transactions))
GROUP BY p.prod_cat
ORDER BY return_value ASC;    -- most negative = largest returns
```

RESULT ON THE REAL CASE DATA

	prod_cat	return_value
0	Books	-134,408.88

DA13 Store type selling the maximum products by value and by quantity.

T-SQL (SQL SERVER)

```
SELECT Store_type, SUM(total_amt) AS sales_value, SUM(Qty) AS quantity,
       RANK() OVER (ORDER BY SUM(total_amt) DESC) AS rank_by_value,
       RANK() OVER (ORDER BY SUM(Qty) DESC)       AS rank_by_qty
FROM Transactions
GROUP BY Store_type
ORDER BY sales_value DESC;
```

RESULT ON THE REAL CASE DATA

	Store_type	sales_value	quantity	rank_by_value	rank_by_qty
0	e-Shop	19,824,816.05	22,763.00	1	1
1	Flagship store	9,715,688.19	11,133.00	2	3
2	MBR	9,674,486.06	11,194.00	3	2

	Store_type	sales_value	quantity	rank_by_value	rank_by_qty
3	TeleShop	9,364,781.07	10,984.00	4	4

DA14 Categories whose average revenue is above the overall average.

T-SQL (SQL SERVER)

```
SELECT p.prod_cat, ROUND(AVG(t.total_amt), 2) AS avg_revenue
FROM Transactions t
JOIN prod_cat_info p
  ON t.prod_cat_code = p.prod_cat_code AND t.prod_subcat_code = p.prod_sub_cat_code
GROUP BY p.prod_cat
HAVING AVG(t.total_amt) > (SELECT AVG(total_amt) FROM Transactions);
```

RESULT ON THE REAL CASE DATA

	prod_cat	avg_revenue
0	Clothing	2,111.87
1	Electronics	2,189.15
2	Books	2,112.82

DA15 Average and total revenue by sub-category, for the top 5 categories by quantity sold.

T-SQL (SQL SERVER)

```
SELECT p.prod_cat, p.prod_subcat,
       ROUND(AVG(t.total_amt), 2) AS avg_revenue, ROUND(SUM(t.total_amt), 2) AS
total_revenue
FROM Transactions t
JOIN prod_cat_info p
  ON t.prod_cat_code = p.prod_cat_code AND t.prod_subcat_code = p.prod_sub_cat_code
WHERE p.prod_cat IN (SELECT TOP 5 p2.prod_cat
                     FROM Transactions t2
                     JOIN prod_cat_info p2
                       ON t2.prod_cat_code = p2.prod_cat_code AND t2.prod_subcat_code =
p2.prod_sub_cat_code
                     GROUP BY p2.prod_cat
                     ORDER BY SUM(t2.Qty) DESC)
GROUP BY p.prod_cat, p.prod_subcat
ORDER BY p.prod_cat, total_revenue DESC;
```

RESULT ON THE REAL CASE DATA

	prod_cat	prod_subcat	avg_revenue	total_revenue
0	Books	Fiction	2,140.22	2,232,250.28

	prod_cat	prod_subcat	avg_revenue	total_revenue
1	Books	Children	2,136.67	2,211,450.87
2	Books	Non-Fiction	2,129.10	2,137,620.29
3	Books	Comics	2,037.68	2,100,848.10
4	Books	DIY	2,108.37	2,085,180.30
5	Books	Academic	2,125.49	2,055,344.20
6	Clothing	Kids	2,136.60	2,110,959.95
7	Clothing	Women	2,071.79	2,082,147.08
8	Clothing	Mens	2,128.26	2,058,030.46
9	Electronics	Mobiles	2,181.09	2,248,702.63
10	Electronics	Audio and video	2,247.96	2,140,057.92
11	Electronics	Cameras	2,165.88	2,133,390.35
12	Electronics	Personal Appliances	2,170.98	2,110,196.40
13	Electronics	Computers	2,181.75	2,090,116.34
14	Footwear	Kids	2,125.99	2,145,126.56

Showing first 15 of 21 rows.

SQL Case Study (Advanced): Mobile Manufacturer Data Analysis

The database 'Cellphones Information' holds cell phone sales. Create the schema from the creation script provided, then answer 10 questions. Each answer must return a single table and goes between the --Qn--BEGIN and --Qn--END marks of the answer template.

Data

Star schema: DIM_MANUFACTURER, DIM_MODEL, DIM_CUSTOMER, DIM_LOCATION, DIM_DATE and FACT_TRANSACTIONS (IDModel, IDCustomer, IDLocation, Date, TotalPrice, Quantity).

Tool and module

SQL Server (T-SQL)
Data Visualization & Analytics (SQL)

1 List all states with customers who bought cellphones from 2005 till today.

T-SQL (SQL SERVER)

```
SELECT DISTINCT L.State
FROM FACT_TRANSACTIONS T
JOIN DIM_LOCATION L ON T.IDLocation = L.IDLocation
WHERE YEAR(T.Date) >= 2005;
```

RESULT ON THE REAL CASE DATA

	State
0	Arizona
1	California
2	Delhi
3	Haryana
4	Karnataka
5	Maharashtra
6	Maryland

2 Which US state buys the most Samsung phones?

T-SQL (SQL SERVER)

```
SELECT TOP 1 L.State, SUM(T.Quantity) AS Qty
FROM FACT_TRANSACTIONS T
JOIN DIM_LOCATION L     ON T.IDLocation = L.IDLocation
JOIN DIM_MODEL M        ON T.IDModel = M.IDModel
JOIN DIM_MANUFACTURER F ON M.IDManufacturer = F.IDManufacturer
WHERE L.Country = 'US' AND F.Manufacturer_Name = 'Samsung'
GROUP BY L.State
ORDER BY Qty DESC;
```

RESULT ON THE REAL CASE DATA

	State	Qty
0	Arizona	18.00

3 Number of transactions for each model per zip code per state.

T-SQL (SQL SERVER)

```
SELECT M.Model_Name, L.ZipCode, L.State, COUNT(*) AS No_of_Transactions
FROM FACT_TRANSACTIONS T
JOIN DIM_MODEL M    ON T.IDModel = M.IDModel
JOIN DIM_LOCATION L ON T.IDLocation = L.IDLocation
GROUP BY M.Model_Name, L.ZipCode, L.State;
```

RESULT ON THE REAL CASE DATA

	Model_Name	ZipCode	State	No_of_Transactions
0	RAZR V3	94005	California	1
1	RAZR V3	92703	California	1
2	OnePlus 6T	85086	Arizona	2
3	Motorola Z	94005	California	3
4	E1100	400006	Maharashtra	4
5	Galaxy S7	110004	Delhi	2
6	3210	530068	Karnataka	2
7	OnePlus 5T	21648	Maryland	2
8	OnePlus X	92703	California	2
9	iPhone 4	21648	Maryland	4
10	C139	400006	Maharashtra	1
11	3210	21648	Maryland	2

	Model_Name	ZipCode	State	No_of_Transactions
12	Thunderbolt	110004	Delhi	1
13	iPhone 5	21648	Maryland	3
14	iPhone 6	92703	California	2

Showing first 15 of 188 rows.

4 The cheapest cellphone (with its price).

T-SQL (SQL SERVER)

```
SELECT TOP 1 Model_Name, Unit_price
FROM DIM_MODEL
ORDER BY Unit_price ASC;
```

RESULT ON THE REAL CASE DATA

	Model_Name	Unit_price
0	3210	14.00

5 Average price of each model in the top 5 manufacturers by sales quantity, ordered by average price.

T-SQL (SQL SERVER)

```
SELECT M.Model_Name, AVG(T.TotalPrice) AS Avg_Price
FROM FACT_TRANSACTIONS T
JOIN DIM_MODEL M ON T.IDModel = M.IDModel
WHERE M.IDManufacturer IN (SELECT TOP 5 M2.IDManufacturer
                           FROM FACT_TRANSACTIONS T2
                           JOIN DIM_MODEL M2 ON T2.IDModel = M2.IDModel
                           GROUP BY M2.IDManufacturer
                           ORDER BY SUM(T2.Quantity) DESC)
GROUP BY M.Model_Name
ORDER BY Avg_Price;
```

RESULT ON THE REAL CASE DATA

	Model_Name	Avg_Price
0	3210	18.50
1	3310 (3330)	31.45
2	5230	34.09
3	6230 (6233)	55.75

	Model_Name	Avg_Price
4	6010 (6020/6030)	58.50
5	6600	60.00
6	RAZR V3	91.36
7	C200	151.08
8	E250	170.67
9	Droid Bionic	175.44
10	E1100	178.18
11	OnePlus X	184.23
12	C139	185.10
13	OnePlus 2	192.67
14	Galaxy S	198.22

Showing first 15 of 29 rows.

6 Customers and their average spend in 2009, where the average is above 500.

T-SQL (SQL SERVER)

```
SELECT C.Customer_Name, AVG(T.TotalPrice) AS Avg_Spend
FROM FACT_TRANSACTIONS T
JOIN DIM_CUSTOMER C ON T.IDCustomer = C.IDCustomer
WHERE YEAR(T.Date) = 2009
GROUP BY C.Customer_Name
HAVING AVG(T.TotalPrice) > 500;
```

RESULT ON THE REAL CASE DATA

	Customer_Name	Avg_Spend
0	Laurel Reitler	1,528.00
1	Lettie Isenhower	870.00
2	Moon Parlato	823.50
3	Danica Bruschke	760.00
4	Celeste Korando	613.00
5	Shawna Palaspas	569.00

7 Any model in the top 5 by quantity in 2008, 2009 and 2010 simultaneously?

T-SQL (SQL SERVER)

```
SELECT Model_Name FROM (
    SELECT TOP 5 M.Model_Name FROM FACT_TRANSACTIONS T JOIN DIM_MODEL M ON T.IDModel =
M.IDModel
    WHERE YEAR(T.Date) = 2008 GROUP BY M.Model_Name ORDER BY SUM(T.Quantity) DESC) A
INTERSECT
SELECT Model_Name FROM (
    SELECT TOP 5 M.Model_Name FROM FACT_TRANSACTIONS T JOIN DIM_MODEL M ON T.IDModel =
M.IDModel
    WHERE YEAR(T.Date) = 2009 GROUP BY M.Model_Name ORDER BY SUM(T.Quantity) DESC) B
INTERSECT
SELECT Model_Name FROM (
    SELECT TOP 5 M.Model_Name FROM FACT_TRANSACTIONS T JOIN DIM_MODEL M ON T.IDModel =
M.IDModel
    WHERE YEAR(T.Date) = 2010 GROUP BY M.Model_Name ORDER BY SUM(T.Quantity) DESC) C;
```

RESULT ON THE REAL CASE DATA

	Model_Name
0	Droid Bionic

8 Manufacturer with the 2nd highest sales in 2009 and in 2010.

T-SQL (SQL SERVER)

```
WITH S AS (
    SELECT YEAR(T.Date) AS Yr, F.Manufacturer_Name, SUM(T.TotalPrice) AS Sales,
           DENSE_RANK() OVER (PARTITION BY YEAR(T.Date) ORDER BY SUM(T.TotalPrice) DESC)
AS Rnk
    FROM FACT_TRANSACTIONS T
    JOIN DIM_MODEL M        ON T.IDModel = M.IDModel
    JOIN DIM_MANUFACTURER F ON M.IDManufacturer = F.IDManufacturer
    WHERE YEAR(T.Date) IN (2009, 2010)
    GROUP BY YEAR(T.Date), F.Manufacturer_Name)
SELECT Yr, Manufacturer_Name, Sales FROM S WHERE Rnk = 2;
```

RESULT ON THE REAL CASE DATA

	Yr	Manufacturer_Name	Sales
0	2009	Samsung	3,357.00
1	2010	Apple	3,242.00

9 Manufacturers that sold in 2010 but not in 2009.

T-SQL (SQL SERVER)

```sql
SELECT F.Manufacturer_Name
FROM FACT_TRANSACTIONS T JOIN DIM_MODEL M ON T.IDModel = M.IDModel JOIN DIM_MANUFACTURER F
ON M.IDManufacturer = F.IDManufacturer
WHERE YEAR(T.Date) = 2010
EXCEPT
SELECT F.Manufacturer_Name
FROM FACT_TRANSACTIONS T JOIN DIM_MODEL M ON T.IDModel = M.IDModel JOIN DIM_MANUFACTURER F
ON M.IDManufacturer = F.IDManufacturer
WHERE YEAR(T.Date) = 2009;
```

RESULT ON THE REAL CASE DATA

	Manufacturer_Name
0	HTC

10 Top 100 customers: average spend and quantity by year, and % change in spend.

T-SQL (SQL SERVER)

```sql
WITH Top100 AS (
    SELECT TOP 100 IDCustomer FROM FACT_TRANSACTIONS
    GROUP BY IDCustomer ORDER BY SUM(TotalPrice) DESC),
Y AS (
    SELECT T.IDCustomer, YEAR(T.Date) AS Yr, AVG(T.TotalPrice) AS Avg_Spend,
AVG(T.Quantity * 1.0) AS Avg_Qty
    FROM FACT_TRANSACTIONS T JOIN Top100 X ON T.IDCustomer = X.IDCustomer
    GROUP BY T.IDCustomer, YEAR(T.Date))
SELECT C.Customer_Name, Y.Yr, Y.Avg_Spend, Y.Avg_Qty,
       (Y.Avg_Spend - LAG(Y.Avg_Spend) OVER (PARTITION BY Y.IDCustomer ORDER BY Y.Yr))
        * 100.0 / LAG(Y.Avg_Spend) OVER (PARTITION BY Y.IDCustomer ORDER BY Y.Yr) AS
Pct_Change_Spend
FROM Y JOIN DIM_CUSTOMER C ON Y.IDCustomer = C.IDCustomer
ORDER BY C.Customer_Name, Y.Yr;
```

RESULT ON THE REAL CASE DATA

	Customer_Name	Yr	Avg_Spend	Avg_Qty	Pct_Change_Spend
0	Alease Buemi	2007	436.00	1.00	NaN
1	Angella Cetta	2003	170.00	1.00	NaN
2	Angella Cetta	2004	282.50	1.00	66.18
3	Angella Cetta	2007	300.00	1.00	6.19
4	Angella Cetta	2010	173.00	1.00	-42.33
5	Arlene Klusman	2004	665.00	1.00	NaN
6	Arlene Klusman	2006	415.00	1.00	-37.59

	Customer_Name	Yr	Avg_Spend	Avg_Qty	Pct_Change_Spend
7	Arlene Klusman	2008	347.00	1.00	-16.39
8	Arlene Klusman	2010	476.00	1.00	37.18
9	Arlette Honeywell	2004	189.00	1.00	NaN
10	Arlette Honeywell	2005	269.00	1.00	42.33
11	Arlette Honeywell	2006	233.00	1.00	-13.38
12	Arlette Honeywell	2007	76.67	1.00	-67.10
13	Arlette Honeywell	2008	502.00	1.00	554.78
14	Bobbye Rhym	2005	319.00	1.00	NaN

Showing first 15 of 200 rows.

TABLEAU · RESOLUTION

Tableau Case Study: Laptop Sales Strategy (London retail chain)

A London retail chain with 15+ stores sells laptops and accessories. It wants a product strategy and pricing policy that maximises projected 2016 revenue, based on 2015 point-of-sale data.

Data

POS.csv (2015 sales: date, configuration, customer OS X/Y, store postcode, retail price) · Laptops.csv (864 configurations: screen, battery, RAM, CPU, HDD) · Store_Locations.csv (store OS X/Y) · London_postal_codes.csv.

Tool and module

Tableau

Tableau

1 Top 20 configurations sold in 2015 (make '20' dynamic).

TABLEAU STEPS

```
Create Parameter  Top N (integer, default 20)
Create Set on Configuration → Top tab → By field: Top [Top N] by COUNT(Number of Records)
Rows: Configuration (filter: Set = In) · Columns: CNT(POS) · sort descending · Show Parameter
```

EXPECTED RESULT (COMPUTED FROM THE CASE DATA, USE IT TO CHECK YOUR WORK)

Configuration	units
61	825
316	823
72	820
345	817
353	817
207	812
53	810
15	804
340	803

Configuration	units
296	796
63	794
213	792
308	789
149	788
289	787

Showing first 15 of 20 rows.

2 Has the average price of these top 20 varied over time?

TABLEAU STEPS

```
Columns: MONTH(Date) · Rows: AVG(Retail Price) · Filter: Top-N set · Color: Configuration (optional)
```

EXPECTED RESULT (COMPUTED FROM THE CASE DATA, USE IT TO CHECK YOUR WORK)

Date	Retail Price
1	499.03
2	492.07
3	442.18
4	477.33
5	466.68
6	418.41
7	452.66
8	444.11
9	401.84
10	427.89
11	420.14
12	379.03

Insight Average price falls from about 499 in January to about 379 in December, with ups and downs between months. Plan pricing month by month rather than with one annual price.

3 Are average prices of the top 20 consistent across stores?

TABLEAU STEPS

```
Rows: Store Postcode · Columns: AVG(Retail Price) · Filter: Configuration (show as single-value dropdown) + Top-N set
```

EXPECTED RESULT (COMPUTED FROM THE CASE DATA, USE IT TO CHECK YOUR WORK)

Store Postcode	mean Retail Price	std Retail Price	count Retail Price
CR7 8LE	404.30	80.30	436
E2 0RY	451.20	38.50	1630
E7 8NW	403.30	77.00	236
KT2 5AU	453.90	40.50	258
N17 6QA	455.20	42.30	235
N3 1DH	403.20	79.40	147
NW5 2QH	453.40	39.20	1695
SE1 2BN	453.90	39.60	2404
SE8 3JD	454.90	38.90	885
SW12 9HD	454.90	39.60	684
SW18 1NN	455.30	37.90	733
SW1P 3AU	404.60	79.90	3318
SW1V 4QQ	453.10	39.10	2373
W10 6HQ	455.30	39.80	625
W4 3PH	410.20	76.70	338

4 Do sales vary by month or by day of month?

TABLEAU STEPS

```
Sheet A: Columns MONTH(Date), Rows CNT(POS)
Sheet B: Columns DAY(Date) (discrete), Rows CNT(POS)
```

EXPECTED RESULT (COMPUTED FROM THE CASE DATA, USE IT TO CHECK YOUR WORK)

5 Do stores with lower average prices sell more?

TABLEAU STEPS

```
Columns: AVG(Retail Price) · Rows: CNT(POS) · Detail: Store Postcode · Analytics pane: Trend Line
```

EXPECTED RESULT (COMPUTED FROM THE CASE DATA, USE IT TO CHECK YOUR WORK)

```
correlation between store avg price and units: r = 0.12 (p = 0.663)
```

Store Postcode	avg_price	units
SW1P 3AU	452.50	61924
SE1 2BN	520.10	44606
SW1V 4QQ	520.40	43440
NW5 2QH	521.10	31382
E2 0RY	520.20	30682
SE8 3JD	520.80	16572
SW18 1NN	520.40	14066
SW12 9HD	521.30	12676
W10 6HQ	520.10	11719
CR7 8LE	454.40	7831
W4 3PH	452.20	6537
N17 6QA	523.00	4734
KT2 5AU	521.50	4336
E7 8NW	449.20	3793
N3 1DH	452.80	3009

6 Group the 864 configurations into 32 groups (High/Low for 5 parameters) and compare average price.

TABLEAU STEPS

```
For each spec, Calculated Field e.g.  [RAM HL] = IF [RAM (GB)] > {FIXED : MEDIAN([RAM (GB)])} THEN "H" ELSE "L" END
Select the 5 HL fields → right-click → Create → Combined Field (32 combinations)
Rows: Combined Field · Columns: AVG(Retail Price) · sort
```

EXPECTED RESULT (COMPUTED FROM THE CASE DATA, USE IT TO CHECK YOUR WORK)

```
Groups: 32 (letters = Screen, Battery, RAM, CPU, HDD)
```

Group	mean	count
HHHHH	792.80	1285
HHHLH	756.10	2509
HHHHL	725.10	1272
HLHHH	704.20	2417
LHHHH	696.60	1251
HHHLL	690.30	2357
HLHLH	661.40	5556
LHHLH	649.10	2916
HHLHH	643.80	2717
HLHHL	629.20	3272
LHHHL	619.00	1692
HHLLH	602.90	6997
LLHHH	600.20	4098
HLHLL	596.10	8225
LHHLL	588.30	4478

Showing first 15 of 32 rows.

7 On average, how far do customers travel to buy laptops (overall and per store)?

TABLEAU STEPS

```
Join POS to Store_Locations on Store Postcode
Calculated Field  Distance = SQRT(SQUARE([C OSx]-[St OS X]) + SQUARE([C OSy]-[St OS Y]))
Overall: AVG(Distance) in a text table · Per store: Rows Store Postcode, Columns
AVG(Distance)
```

EXPECTED RESULT (COMPUTED FROM THE CASE DATA, USE IT TO CHECK YOUR WORK)

```
Answer: overall average distance = 3,681 m (OS grid units are metres)
```

Store Postcode	Distance_m
SW1P 3AU	2,728.00
E7 8NW	2,919.00
CR7 8LE	3,031.00
SW1V 4QQ	3,118.00
SE1 2BN	3,152.00
NW5 2QH	3,529.00
KT2 5AU	3,616.00
E2 0RY	3,799.00
W10 6HQ	4,208.00
SE8 3JD	5,198.00
W4 3PH	5,217.00
SW12 9HD	5,299.00
N17 6QA	5,318.00
N3 1DH	5,375.00
SW18 1NN	6,649.00

8 Does store proximity help increase sales?

TABLEAU STEPS

```
Scatter: Columns AVG(Distance), Rows CNT(POS), Detail Store Postcode, add Trend Line and
```

```
read R²/p-value
```

EXPECTED RESULT (COMPUTED FROM THE CASE DATA, USE IT TO CHECK YOUR WORK)

```
r = -0.51, p = 0.052
```

Store Postcode	avg_dist	units
CR7 8LE	3,031.00	7831
E2 0RY	3,799.00	30682
E7 8NW	2,919.00	3793
KT2 5AU	3,616.00	4336
N17 6QA	5,318.00	4734
N3 1DH	5,375.00	3009
NW5 2QH	3,529.00	31382
SE1 2BN	3,152.00	44606
SE8 3JD	5,198.00	16572
SW12 9HD	5,299.00	12676
SW18 1NN	6,649.00	14066
SW1P 3AU	2,728.00	61924
SW1V 4QQ	3,118.00	43440
W10 6HQ	4,208.00	11719
W4 3PH	5,217.00	6537

9–10 Store-view dashboard with interactive store filter, then a Story.

TABLEAU STEPS

```
Dashboard: drag sheets 3, 5, 7 and 8 → Use as Filter on the store sheet (funnel icon) or a
Store Postcode quick filter applied to All Using This Data Source
Story: New Story → one story point per finding (Top configurations → Price trend → Store
pricing → Distance) with a caption headline each
```

EXPECTED RESULT (COMPUTED FROM THE CASE DATA, USE IT TO CHECK YOUR WORK)

```
Build in Tableau: no numeric answer
```

TABLEAU · RESOLUTION

Tableau Case Study: Sports Equipment Retailer

A global sports-equipment retailer's product manager wants to analyse revenue across retailer countries, products and time. Submit a .twbx with one worksheet per question.

Data

Sales Data (16,210 orders, 2012–2014: retailer country/type, order mode, product category/type/product, revenue, quantity, gross margin).

Tool and module

Tableau

Tableau

1 Market share of revenue by retailer country, with labels and no legend.

TABLEAU STEPS

```
Marks: Pie · Angle: SUM(Revenue) → Quick Table Calculation → Percent of Total
Color + Label: Retailer Country and % · right-click legend → Hide Card
```

EXPECTED RESULT (COMPUTED FROM THE CASE DATA, USE IT TO CHECK YOUR WORK)

Retailer_Country	share_%
Canada	25.61
United Kingdom	22.55
France	21.96
Singapore	17.00
Australia	12.89

2 Revenue vs quantity by product: is there a correlation?

TABLEAU STEPS

```
Columns: SUM(Quantity) · Rows: SUM(Revenue) · Detail: Product · Analytics → Trend Line;
put the inference in the Caption
```

EXPECTED RESULT (COMPUTED FROM THE CASE DATA, USE IT TO CHECK YOUR WORK)

```
r = -0.09, p = 0.3731
```

3 Quarterly revenue across the three years for each country.

TABLEAU STEPS

```
Columns: YEAR(Order Date), QUARTER(Order Date) · Rows: Retailer Country, SUM(Revenue) (or a highlight table)
```

EXPECTED RESULT (COMPUTED FROM THE CASE DATA, USE IT TO CHECK YOUR WORK)

Year	2012				2013	
Order_Date	1	2	3	4	1	2
Retailer_Country						
Australia	4,407,119.00	3,931,133.00	3,962,447.00	3,846,117.00	7,407,684.00	8,214
Canada	11,498,061.00	10,405,175.00	11,035,589.00	8,824,720.00	16,119,670.00	12,77
France	9,412,355.00	9,673,826.00	10,869,909.00	9,097,834.00	11,822,896.00	11,77
Singapore	7,926,369.00	6,282,255.00	8,208,621.00	8,179,774.00	8,776,506.00	8,607
United Kingdom	9,246,476.00	10,739,814.00	9,797,027.00	9,700,637.00	11,415,094.00	12,43

4 Dual-axis chart of revenue and quantity by product type.

TABLEAU STEPS

```
Columns: Product Type · Rows: SUM(Revenue), SUM(Quantity) → right-click 2nd → Dual Axis → set marks Bar + Line
```

EXPECTED RESULT (COMPUTED FROM THE CASE DATA, USE IT TO CHECK YOUR WORK)

5 Set of top 5 product types by quantity: what happens to their revenue?

TABLEAU STEPS

```
Create Set on Product Type → Top 5 by SUM(Quantity) · Columns: IN/OUT(Set) · Rows: SUM(Revenue) · Color: Set
```

EXPECTED RESULT (COMPUTED FROM THE CASE DATA, USE IT TO CHECK YOUR WORK)

```
Top 5 by quantity: ['Cooking Gear', 'Climbing Accessories', 'Knives', 'Lanterns',
'Sunscreen']
InTop5
False    76.4
True     23.6
```

6 Total revenue and profit by retailer country (Profit = Revenue × Gross margin).

TABLEAU STEPS

```
Calculated Field  Profit = [Revenue] * [Gross margin] · Rows: Retailer Country · Columns:
Measure Values (Revenue, Profit)
```

EXPECTED RESULT (COMPUTED FROM THE CASE DATA, USE IT TO CHECK YOUR WORK)

Retailer_Country	Revenue	Profit
Canada	144,362,366.00	61,528,897.00
United Kingdom	127,116,809.00	54,106,135.00
France	123,790,664.00	52,688,950.00
Singapore	95,829,367.00	40,641,427.00
Australia	72,668,227.00	30,937,117.00

7–8 Top 15 products sold in 2014, with a dynamic year and a dynamic Top N.

TABLEAU STEPS

```
Parameters: Select Year (2012–2014), Top N (default 15)
Filter: YEAR([Order Date]) = [Select Year] (calculated boolean)
Product filter → Top → By field: Top [Top N] by SUM(Quantity) · add to Context so the year
applies first
```

EXPECTED RESULT (COMPUTED FROM THE CASE DATA, USE IT TO CHECK YOUR WORK)

Product	Quantity
Granite Carabiner	308933
TrailChef Water Bag	243293
Single Edge	230068

Product	Quantity
TrailChef Kettle	127000
Firefly Rechargeable Battery	124833
TrailChef Cup	116117
Pocket Gizmo	73411
Course Pro Gloves	69557
Granite Axe	69500
Canyon Mule Cooler	68076
Glacier Basic	67403
EverGlow Lamp	65306
Course Pro Umbrella	55127
Course Pro Golf and Tee Set	53679
TrailChef Canteen	50582

9 Pareto chart of quantity by product type; colour the types making up 80% of quantity.

TABLEAU STEPS

```
Columns: Product Type (sort desc by SUM(Quantity)) · Rows: SUM(Quantity), SUM(Quantity) →
2nd: Quick Table Calc → Running Total → Add Secondary → Percent of Total
Dual Axis (bar + line) · Calculated Field  In80 =
RUNNING_SUM(SUM([Quantity]))/TOTAL(SUM([Quantity])) -
SUM([Quantity])/TOTAL(SUM([Quantity])) < 0.8 → Color
```

EXPECTED RESULT (COMPUTED FROM THE CASE DATA, USE IT TO CHECK YOUR WORK)

```
Types making up 80%: ['Cooking Gear', 'Climbing Accessories', 'Knives', 'Lanterns',
'Sunscreen', 'Insect Repellents', 'Sleeping Bags', 'Tools', 'Golf Accessories']
```

10 Year-on-year % growth by product category.

TABLEAU STEPS

```
Columns: YEAR(Order Date) · Rows: Product Category · Text: SUM(Revenue) → Quick Table Calc
→ Year over Year Growth
```

EXPECTED RESULT (COMPUTED FROM THE CASE DATA, USE IT TO CHECK YOUR WORK)

Year	2012	2013	2014
Product_Category			
Camping Equipment	NaN	25.10	-25.00
Golf Equipment	NaN	38.60	-22.60
Mountaineering Equipment	NaN	50.90	-7.40
Outdoor Protection	NaN	-58.60	-55.70
Personal Accessories	NaN	46.00	-20.80

11 3-month moving average of revenue vs actual by month.

TABLEAU STEPS

```
Columns: MONTH(Order Date) (continuous) · Rows: SUM(Revenue) twice → 2nd: Quick Table Calc
→ Moving Average → Edit: previous 2 + current → Dual Axis, Synchronize
```

EXPECTED RESULT (COMPUTED FROM THE CASE DATA, USE IT TO CHECK YOUR WORK)

12 View showing total by country for a user-selected measure (Revenue, Profit or Quantity).

TABLEAU STEPS

```
Parameter  Select Measure (string list: Revenue, Profit, Quantity)
Calculated Field  Measure = CASE [Select Measure] WHEN "Revenue" THEN SUM([Revenue]) WHEN
"Profit" THEN SUM([Profit]) ELSE SUM([Quantity]) END
Rows: Retailer Country · Columns: [Measure] · Show Parameter
```

EXPECTED RESULT (COMPUTED FROM THE CASE DATA, USE IT TO CHECK YOUR WORK)

	Revenue	Profit	Quantity
Retailer_Country			
Australia	72,668,227.00	30,937,117.00	1526686
Canada	144,362,366.00	61,528,897.00	2948790

Retailer_Country	Revenue	Profit	Quantity
France	123,790,664.00	52,688,950.00	2648399
Singapore	95,829,367.00	40,641,427.00	2032000
United Kingdom	127,116,809.00	54,106,135.00	2692585